天津工业大学软件学院

实训总结报告



项目名称：CodePilot—AI助力程序员成长平台

合作培养企业：天津市融创软通科技股份有限公司

专业方向：大模型技术应用

项目负责人：马文勇

专业班级：融创软通-大模型技术应用-23级工大班

姓　　名：徐晶睿

实训起止日期：2026.6.22—2026.7.17

1 引言

1.1 项目概述

随着人工智能技术的飞速发展，大语言模型（Large Language Model, LLM）在代码理解、错误分析和知识点梳理方面展现出卓越能力。与此同时，在线判题系统（Online Judge, OJ）作为计算机专业教学和编程竞赛的核心工具，在国内外拥有庞大的用户群体。如何将 AI 能力与传统 OJ 系统有机结合，在不破坏学习效果的前提下为用户提供智能化、个性化的学习辅助，成为当前计算机教育领域的一个重要研究方向。

CodePilot（码途）正是在此背景下诞生的综合性实训项目。本项目旨在开发一个集 ACM/OJ 在线判题、AI Agent 智能分析、模拟面试与求职辅助于一体的编程竞赛与学习平台。项目涵盖了从前端到后端、从数据库到容器化、从传统判题引擎到 LLM Agent 工作流编排的完整技术栈，是对团队全栈开发能力、系统设计能力和 AI 技术应用能力的综合实践。

本项目的核心设计理念是“Agent 不在做题时介入，只在事后分析和求职推荐”——AI 扮演的是“教练”而非“枪手”的角色。系统在用户提交代码并获得评测结果后，由 AI Agent 自动进行代码审查、错误复盘、知识点标注和学习建议生成，同时基于用户长期的做题数据建立多维度用户画像，进而提供个性化的面试推荐、比赛推荐和简历生成服务，形成“刷题→记录过程→Agent 事后分析→建立用户画像→推荐面试/比赛→模拟面试→生成简历→求职”的完整闭环。

项目参考了 LeetCode、洛谷、AcWing、NowCoder 等国内外主流 OJ 和求职平台的优秀设计理念，并基于 Dify 工作流平台（Docker 自部署）实现 AI Agent 能力的可视化编排，调用智谱 GLM-4-Flash 大模型提供智能分析服务。

1.2 需求分析

随着计算机专业毕业生数量的持续增长和互联网行业技术面试门槛的不断提高，编程学习者面临着以下几个核心痛点：

（1）缺乏系统化的训练路径：传统刷题方式缺乏针对性指导，学习者难以准确评估自身水平，不知道从何学起、哪里薄弱。

（2）错误反馈不够智能：现有 OJ 平台仅返回“通过”或“不通过”的简单判题结果，缺乏对代码质量、算法效率、错误原因的深度分析。

（3）学习过程无法沉淀：用户的每次提交、每道题的思考过程、每次错误的修改轨迹等宝贵数据未被有效记录和利用。

（4）刷题与求职脱节：学习者在 OJ 平台的刷题数据无法直接转化为求职竞争力，缺乏从刷题到求职的完整衔接。

CodePilot（码途）项目正是为了解决上述痛点而设计，其实际价值体现在以下几个方面：

在学习辅助层面，系统通过做题过程全记录（代码版本、判题结果、思考时间、修改轨迹、个人笔记）和 AI Agent 事后分析（代码 Review、解法对比、错误复盘、知识点自动标注），为学习者提供精准的个性化反馈和系统化的训练路径。

在求职衔接层面，系统通过用户画像（算法能力、编程习惯、学习行为、错误模式、成长趋势等七个维度）实现从刷题数据到求职能力的转化，提供模拟面试（算法面试、系统设计面试、HR 行为面试）、简历自动生成和岗位智能推荐等一站式求职服务。

在平台体验层面，系统提供 ACM 赛制的竞赛功能（实时排名、Rating 计算、比赛聊天室、赛后复盘）、修仙风格的等级系统（练气期→筑基期→金丹期→元婴期→化神期→渡劫期→大乘期→飞升期）、知识图谱导航和社交互动功能，打造沉浸式的编程学习社区体验。

在技术实践层面，本项目为团队成员提供了完整的大型 Web 应用开发经验，涵盖了 FastAPI 异步后端开发、Vue3 前端工程化、Docker 容器化部署、MySQL/Neo4j/Redis 多数据库协同、Dify 工作流 AI Agent 编排、判题沙箱安全隔离等前沿技术方向。

1.3 运行环境

（1）硬件环境

• 开发机：CPU Intel Core i5 / AMD Ryzen 5 及以上；内存 16GB 及以上；硬盘 512GB SSD 及以上

• 服务器：CPU 4 核及以上；内存 8GB 及以上；硬盘 100GB 及以上；网络需具备公网 IP 及域名

• 容器环境：支持 Docker 24.0+ 和 docker-compose v2.0+

（2）软件环境

• 操作系统：Windows 11 / Linux (Ubuntu 22.04+) / macOS

• 后端框架：Python 3.13 + FastAPI 0.100+（异步高性能 Web 框架）

• 数据库：MySQL 8.0（主库）+ Neo4j 5.0（知识图谱）+ Redis 7.0（缓存/消息队列/判题队列）

• ORM 与迁移：SQLAlchemy 2.0 + Alembic

• 任务队列：Celery + Redis（判题等异步任务处理）

• 前端框架（Web）：Vue3 + TypeScript + Vite

• UI 组件库：Element Plus（企业级 UI 组件）

• 代码编辑器：Monaco Editor（VS Code 核心编辑器）

• AI Agent 平台：Dify（Docker 自部署，可视化 Workflow + Chatflow 编排）

• 大语言模型：智谱 GLM-4-Flash（通过 Dify 配置调用）

• 判题沙箱：Docker 临时容器 + seccomp 安全配置 + 网络隔离 + 磁盘配额

• 容器编排：Docker + docker-compose（单机部署与服务编排）

• 版本控制：Git + Gitee（团队协作与代码管理）

• 测试框架：pytest + pytest-asyncio（单元测试与异步测试）

• 代码规范：ruff（代码格式化）+ mypy（静态类型检查）

• 开发工具：VS Code（集成开发环境）

2 项目设计

2.1 设计思路

2.1.1 总体架构设计

"CodePilot（码途）"整体采用前后端分离的 B/S 架构：后端基于 FastAPI 异步框架统一提供 RESTful API（/api/v1）与 WebSocket 服务；前端拆分为 Web 用户端与 Admin 管理端两个独立 Vue 3 工程，共用同一后端与账号体系。本人负责管理端整体（前端 13 个页面 + 后端 /admin 接口组）、好友与聊天（用户端页面 + 后端 /social 接口组 + /ws/chat 实时通道）、个人信息页面（前端 + 联调）以及上述模块的前后端联调。主要技术选型如下：

表 1 技术选型表

层次	技术选型	选型理由
前端	Vue 3 + TypeScript + Vite	组合式 API + 类型检查，联调期即可暴露字段错误
UI / 图表	Element Plus + ECharts	后台组件齐全，仪表盘图表成熟
状态管理	Pinia	聊天会话、未读数、在线状态等跨组件共享
HTTP 通信	Axios（拦截器）	统一实现 Token 携带、401 刷新、错误处理
实时通信	原生 WebSocket	聊天需服务端主动推送，与 FastAPI 天然适配
后端	FastAPI + SQLAlchemy 2.0（asyncio）	异步 IO 兼顾高并发 API 与 WS 长连接；自动生成的 Swagger 文档即联调契约
存储	MySQL + Redis + MinIO	关系数据持久化；在线状态/Token 黑名单（带 TTL）；头像对象存储

2.1.2 后端分层架构设计

后端严格遵循"API 层 → Service 层 → Repository 层 → Model 层"四层

单向依赖：API 层只做路由、参数校验与鉴权依赖注入（get_current_user / require_admin）；Service 层编排业务逻辑（如互关交集计算、消息落库+推送）；Repository 层封装数据访问；Model 层为 SQLAlchemy ORM。全项目统一响应包装 {code, message, data}、统一异常处理与中间件（CORS、限流、日志），这些约定是前后端联调的契约基础。

2.1.3 管理端设计思路

管理端解决的核心问题是"在不影响用户端的前提下提供完整、安全的管理能力"，设计上有四个关键点：

1. 独立工程、共享后端：管理端独立构建部署，与用户端互不影响，但复用同一套后端服务与账号体系，避免重复建设；

2. 基于角色的访问控制（RBAC）：users 表以 role 字段区分 user/admin，后端管理接口统一挂在 /api/v1/admin 下并由 require_admin 依赖强制鉴权（非管理员返回 403），前端在登录时与路由守卫中双重校验，形成"体验层拦截 + 安全层拦截"的双保险；

3. 功能围绕运营闭环设计：按"看数据 → 管内容 → 管评测 → 配系统 → 查日志"组织菜单，侧边栏分为仪表盘 / 内容管理 / 用户与社区 / 评测与审核 / 系统管理五组；

4. 重内容场景体验优化：题目管理内嵌 Markdown 编辑器、测试数据支持 JSON/zip 批量导入、列表页统一"筛选区 + 分页表格 + 对话框表单"范式。

2.1.4 好友功能设计思路

好友功能的核心决策是采用"单向关注 + 互关即好友"的关注制模型，替代传统"好友申请—审批"模型：

表 2 关注制模型

对比维度	申请—审批模型	关注制模型（本项目采用）
数据表	申请表 + 好友关系表（2 张）	仅 user_follows 一张表
状态机	待处理/已同意/已拒绝，流转复杂	无状态，关注/取关为开关操作
用户链路	申请后等待对方同意，链路长	即搜即关注，互关即成好友，链路短
解除关系	需专门删除接口	取关即解除

在该模型下，"好友"不是存储出来的关系，而是"我关注的人 ∩ 关注我的人"实时算出的交集，数据零冗余，且天然兼容"单向关注"与"双向好友"两种社交形态。

2.1.5 聊天功能设计思路

聊天功能围绕"实时性、可靠性、可维护性"设计，有五个核心决策：

1. WebSocket 为主、REST 兜底：实时收发走 WS 长连接（服务端可推送到对方所有在线连接，支持多端同步）；历史消息、会话列表等请求—响应语义

操作用 REST；WS 断开时发送自动降级为 REST，消息不丢失；

2. 三表数据模型：会话表、会话成员表、消息表分离，type 字段预留 group，私聊与未来群聊共用一套模型；

3. 基于时间戳的已读设计：不在消息表逐条维护 is_read，而是在会话成员表维护 last_read_at，未读数 ="晚于该时间且非本人发送"的消息计数，读操作写开销从 O(n) 降为 O(1)；

4. 基于 Redis TTL 的在线状态：online:{user_id} 键 TTL 300 秒、心跳续期，异常断连到期自动回收，无需清理任务；

5. 心跳保活 + 断线重连：60 秒 ping/pong 兼作 NAT 保活与在线续期；非主动断开按 3s/6s/9s… 线性退避重连，最多 5 次。

2.1.6 个人信息页面设计思路

1. 读写分离的表设计：账号核心字段存 users 窄表（支撑高频登录鉴权查询），15 个展示型扩展字段存一对一的 user_profiles 宽表（支持 JSON 字段）；

2. 局部更新的编辑接口：PUT /user/profile 全字段可选，只更新非空字段，避免并发覆盖；positions 更新时自动同步 target_position 主岗位；

3. 带降级的头像上传链路：前端校验（jpg/png ≤10MB）→ 后端二次校验 → 删旧头像 → 上传 MinIO → 回写 avatar_url，MinIO 不可用时降级本地静态目录；

4. 游戏化等级体系：Rating 映射"练气期→飞升期"八级修仙徽章并展示升级进度条，配合 52 周活跃度热力图与成就墙增强成长感。

2.1.7 前后端联调设计思路

联调遵循"契约先行、并行开发、逐口联调"：事先约定统一响应结构、统一分页（page/page_size，data 固定 {list, total} 结构）、JWT 双 Token 认证、WebSocket 消息协议（8 种消息类型及字段）；前端 axios 请求拦截器统一携带 Token，响应拦截器用"刷新锁 + 等待队列"实现 401 单飞刷新重放（并发只刷新一次）；浏览器 WS 无法自定义请求头，约定 JWT 经 query 参数携带，服务端 accept 前验签；开发环境用 Vite 代理转发 /api、/ws、/static 解决跨域。

2.2 模块功能介绍

2.2.1 管理端模块

管理端共 13 个页面（1 个登录页 + 12 个管理页），统一在"分组侧边栏 + 主内容区"的后台布局下使用：

表 3 后台布局表

页面	功能
管理员登录	登录后校验 role==admin，非管理员拒绝进入；Token 持久化、自动刷新、退出登录

页面	功能
数据仪表盘	平台核心指标统计卡 + ECharts 趋势图 + 最近活动列表
题目管理	条件筛选；MdEditor 编辑题面（即时预览）；测试数据 JSON/zip 批量上传；删除
比赛管理	比赛 CRUD：标题、赛制（ACM/OI/IOI）、起止时间、规则、关联题目
知识点管理	知识点 CRUD（名称/描述/难度/分类），预留前后置关系维护
职位管理	岗位 CRUD；上架/下架开关；薪资区间交叉校验
用户管理	筛选列表；创建/编辑用户；禁用用户（必填原因，禁用后无法登录）
公告通知	发布公告（实时推送用户端，返回送达人数）、列表、撤回
提交管理	提交列表筛选、详情抽屉（完整代码/判题结果）、重新判题（带在途保护）
违规审核	违规记录筛选、证据详情查看、审核判定（确认违规/误报 + 备注）
代码查重	统计卡、手动触发检测、相似提交对及相似度详情、报告审核
Agent 配置	通用/模型/工作流三页签；工作流逐个启停，状态持久化
日志审计	操作/对话/判题/LLM 调用四类日志的筛选与分页查询

管理端页面路由与后端接口的对应关系如下表（前缀 /api/v1，全部要求管理员身份）：

表 4 管理端路由表

页面（路由）	调用的后端接口
登录（/login）	POST /auth/login、POST /auth/token/refresh、POST /auth/logout
仪表盘（/）	GET /admin/dashboard/overview
题目管理（/problems）	GET/POST /admin/problems、GET/PUT/DELETE /admin/problems/{id}、POST /admin/problems/{id}/testcases
比赛管理（/contests）	GET/POST /admin/contests、PUT/DELETE /admin/contests/{id}
知识点管理（/knowledge）	GET/POST /admin/knowledge/points、PUT/DELETE /admin/knowledge/points/{id}、POST /admin/knowledge/relations
职位管理（/jobs）	GET/POST /admin/jobs、PUT/DELETE /admin/jobs/{id}、PUT /admin/jobs/{id}/toggle
用户管理（/users）	GET/POST /admin/users、PUT /admin/users/{id}、PUT /admin/users/{id}/disable、DELETE /admin/users/{id}
公告通知（/announcements）	POST/GET /admin/announcements、DELETE /admin/announcements/{id}
提交管理（/submissions）	GET /admin/submissions、GET /admin/submissions/{id}、POST /admin/submissions/{id}/rejudge
违规审核（/violations）	GET /admin/violations、PUT /admin/violations/{id}/review

页面（路由）	调用的后端接口
代码查重（/plagiarism）	POST /admin/plagiarism/detect、GET /admin/plagiarism/reports[/{id}]、PUT /admin/plagiarism/reports/{id}/review
Agent 配置（/agent-config）	GET/PUT /admin/agent/config、GET /admin/agent/workflows、PUT /admin/agent/workflows/{id}/toggle
日志审计（/logs）	GET /admin/logs/operations、GET /admin/logs/conversations、GET /admin/logs/judges、GET /admin/logs/llm

2.2.2 好友功能模块

好友功能围绕"找人 → 关注 → 成好友 → 开聊"的用户路径设计：

表 5 好友功能设计路径

功能	说明
用户搜索（添加好友入口）	用户名/昵称模糊搜索（排除自己），搜索框 300ms 防抖，结果卡片一键关注
关注/取关	开关式（toggle）接口，返回最新状态；禁止关注自己；关注成功即向对方写入"新粉丝"通知
关注/粉丝列表	同一页面 Tab 切换，分页展示（头像、昵称、Rating、关注时间、关注状态）
互关好友列表	"我关注的 ∩ 关注我的"集合交集实时计算，作为聊天页联系人数据源
他人主页	展示公开资料与最近提交（不含邮箱等私有字段），提供"关注""发起聊天"按钮

好友模块接口清单（前缀 /api/v1，均需登录）：

表 6 好友模块接口

方法	路径	功能
POST	/social/follow/{user_id}	关注/取关开关
GET	/social/followers	粉丝列表（分页）
GET	/social/following	关注列表（分页）
GET	/social/friends	互关好友列表（分页）
GET	/social/users/search	搜索用户（keyword 模糊匹配）
GET	/user/{user_id}/profile	他人公开主页资料
GET	/user/{user_id}/submissions	他人最近提交记录（分页）

好友关系仅由关注关系表 user_follows（follower_id、following_id、created_at）一张表支撑，表结构详见 3.2 节数据库设计。

2.2.3 聊天模块

聊天模块在好友体系之上实现实时通信，页面为三栏布局（联系人列表 | 聊

天区 | 对方资料卡）：

表 7 聊天模块布局

功能	说明
会话列表	对方信息、最近消息摘要、未读角标、在线绿点，按最近消息倒序；已有会话与互关好友合并去重
发起私聊	与对方的会话已存在则自动复用，否则创建新会话
实时消息收发	WebSocket 长连接收发;发送成功服务端回执并推送对方所有在线连接（支持多端同步）
多消息类型	文本；代码（深色代码块 + 一键复制）；题目链接（题目卡片）
历史消息	REST 分页加载，按时间正序渲染，按日期分组显示分割线
已读回执	打开会话更新本人 last_read_at 并 WS 通知对方；未读数实时清零
在线状态	连接建立推送在线快照，好友上下线实时广播，联系人绿点渲染
连接管理	60 秒心跳保活（兼在线续期）；断线按线性退避重连最多 5 次；WS 不可用时发送降级 REST
全局未读角标	顶部导航栏展示全部会话总未读数

聊天 REST 接口清单（前缀 /api/v1/social）：

表 8 聊天相关接口

方法	路径	功能
GET	/conversations	会话列表（含 target_user、last_message、unread_count）
POST	/conversations	创建/获取私聊会话（body: target_user_id）
GET	/conversations/{id}/messages	历史消息（成员鉴权，分页）
POST	/conversations/{id}/messages	发送消息（REST 兜底）
POST	/conversations/{id}/read	标记会话已读
GET	/online-status/{user_id}	查询用户在线状态
GET	/unread-count	当前用户总未读数

WebSocket 协议（WS /ws/chat?token=JWT）：

表 9 Websocket 设计

消息类型	方向	说明
message	客户端 → 服务端	发送消息（conversation_id、content、message_type）
message_sent	服务端 → 客户端	发送成功回执（含服务端生成的消息 ID 与时间）
message	服务端 → 客户端	收到新消息（冗余发送者昵称/头像，免二次查询）
read	双向	标记已读 / 通知对方已读
online / offline	服务端 → 客户端	好友上线 / 下线广播
online_snapshot	服务端 → 客	连接建立时推送全量在线用户列表

	户端	
ping / pong	双向	心跳保活与在线状态续期（60 秒一次）

聊天数据由会话表、会话成员表（含 last_read_at 已读时间戳）、消息表三张表支撑，表结构详见 3.2 节。未读数计算规则：某会话未读数 = 该会话中 created_at 晚于 last_read_at 且非本人发送的消息条数。

2.2.4 个人信息页面模块

个人信息页面（/profile）采用"顶部资料卡 + 中部活跃度区 + 底部编辑与成就区"布局：

表 10 个人信息页面设计

功能	说明
资料展示	头像、昵称、管理员徽标、目标职位@目标公司；Rating、全站排名、解题数、关注数、粉丝数（可点击跳转列表页）
修仙等级	Rating 映射"练气期→飞升期"八级徽章，展示到下一级的进度条
资料编辑	昵称、简介、学校、常用语言、求职意向等字段局部更新；后端未支持的字段自动降级重试
头像上传	点击头像触发上传（jpg/png ≤10MB），MinIO 存储 + 本地降级，成功后页面与顶导头像即时刷新
活跃度热力图	52 周 ×7 天提交活跃度展示
成就与统计	成就徽章墙；解题数、全球排名、连续打卡等统计

个人信息模块接口清单（前缀 /api/v1）：

表 11 个人信息页面接口

方法	路径	功能
GET	/user/profile	获取本人完整资料
PUT	/user/profile	更新资料（全字段可选，局部更新）
POST	/user/avatar	上传头像（multipart，字段名 file）
GET	/user/stats	用户统计（解题数/排名/打卡等）
GET	/user/activity-heatmap	52 周活跃度热力图数据
GET	/user/achievements	成就徽章列表
GET	/social/following、/social/followers	关注数 / 粉丝数（取分页 total）

用户数据由 users 主表（账号核心字段）与 user_profiles 扩展表（展示型字段，一对一）共同支撑，表结构详见 3.2 节。

2.2.5 前后端联调工作

表 12 前后端联调项

联调项	具体内容

联调项	具体内容
接口契约对齐	统一响应 {code, message, data}；分页统一 page/page_size 与 {list, total} 结构；时间字段统一 ISO 8601；枚举值统一（消息类型、用户状态、角色）
认证与 Token 刷新	请求拦截器统一附加 Bearer Token；401 时"刷新锁 + 等待队列"单飞刷新并重放原请求；刷新失败清会话跳登录页
WebSocket 联调	约定 query 参数鉴权；逐条联调 8 种消息类型的字段与触发时机；联调心跳间隔、重连策略、多端在线同步
权限联调	逐接口验证非管理员调管理接口返回 403、前端拦截跳登录；验证聊天接口的会话成员鉴权
跨域与代理	Vite 代理 /api、/static 至 8000 端口，/ws 开启 WebSocket 代理
文件上传联调	约定 multipart 字段名、格式与大小限制、响应字段 avatar_url 及 URL 拼接规则
问题排查修复	定位并修复分页参数不一致、Token 并发重复刷新、WS 重连未读数不同步等联调问题（详见第 5 章）

2.3 模块结构图

系统总体结构：本人负责部分横跨三个物理部分——管理端 Admin 工程（13 个页面）、用户端 Web 工程（好友/聊天/个人信息三模块）、后端 FastAPI 服务（auth/social/user/admin 四组接口 + /ws/chat 网关），底层由 MySQL、Redis、MinIO 支撑，如图 1 系统总体结构图所示。

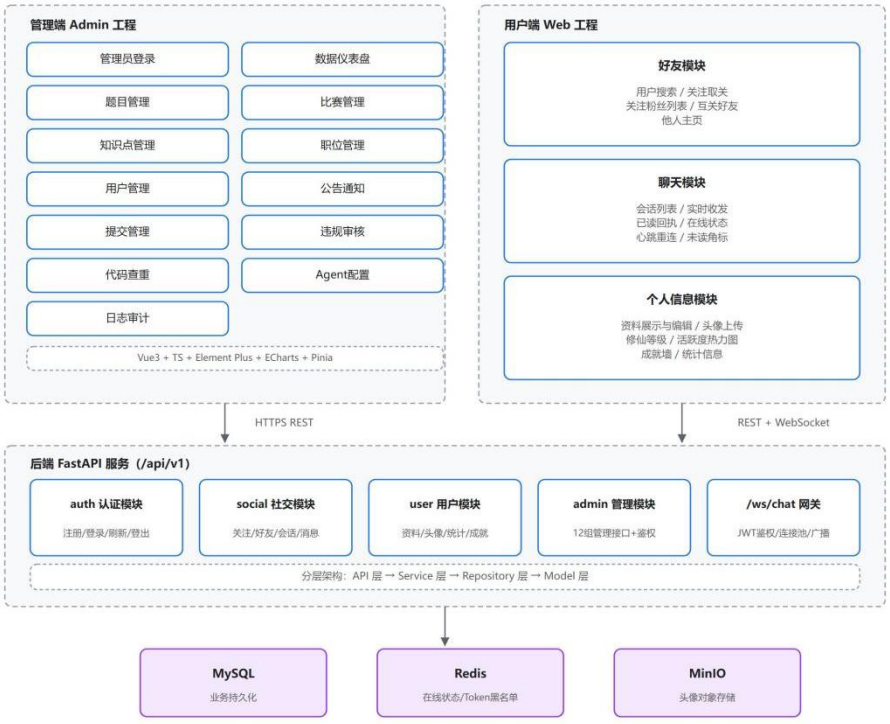


图 1 系统总体结构图

后端分层结构：后端按 API 层 → Service 层 → Repository 层 → Model

层四层单向依赖组织，层间通过依赖注入解耦，如图 2 后端分层结构图所示。

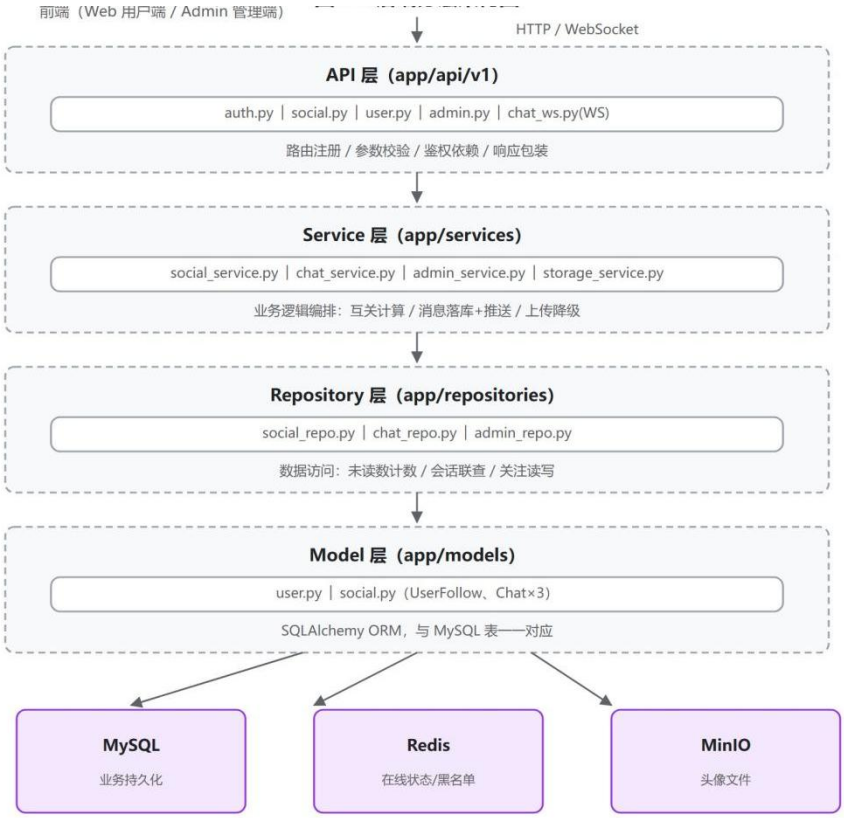


图 2 后端分层结构图

聊天模块前后端协作结构：前端由聊天页面、Pinia 状态管理、API 封装与 WS 客户端组成；后端由 WS 端点（连接池/广播）、聊天 REST 接口、Service 与 Repository 组成，落库 MySQL 聊天三表、在线状态存 Redis，如图 3 聊天模块前后端协作结构图所示。

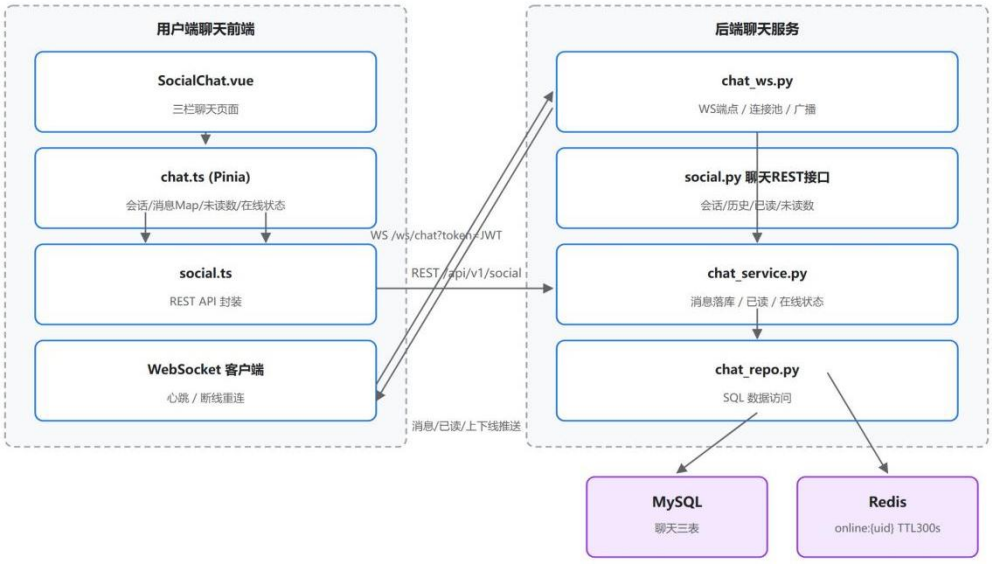


图 3 聊天模块前后端协作结构图

管理端前端结构：登录页独立全屏，12 个管理页挂在 AdminLayout 布局下，

公共基础设施（请求封装、认证 store、路由守卫、TS 类型）支撑全部页面，如图 4 管理端前端结构图所示。

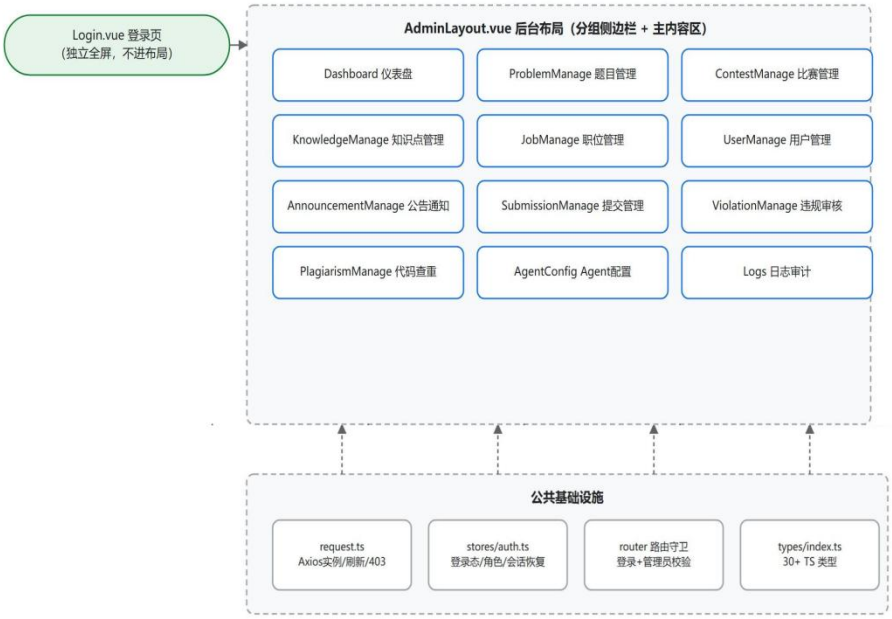


图 4 管理端前端结构图

2.4 程序流程图

以下给出本人负责模块的十个核心程序流程。

（1）管理员登录与权限校验流程：登录页输入账号密码 → 后端校验并签发 JWT 双 Token → 前端检查 role==admin，非管理员清空会话 → 进入后台后请求自动携带 Token → 后端 require_admin 逐接口校验：401 自动刷新重放，403 清会话跳登录，如图 5 管理员登录与权限校验流程图所示。

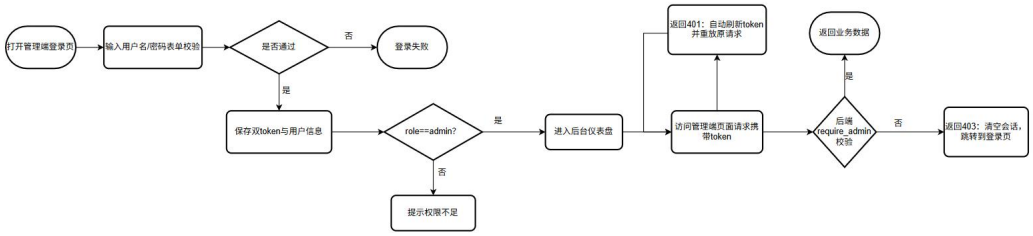


图 5 管理员登录与权限校验流程图

（2）Token 自动刷新流程：请求携带 access_token → 收到 401 后检查本地 refresh_token（无则跳登录）→ 检查是否已有刷新进行中（有则挂入等待队列）→ 加刷新锁调刷新接口 → 成功则更新 Token、释放锁、依次重放队列请求；失败则清会话跳登录，如图 6 Token 自动刷新流程图所示。

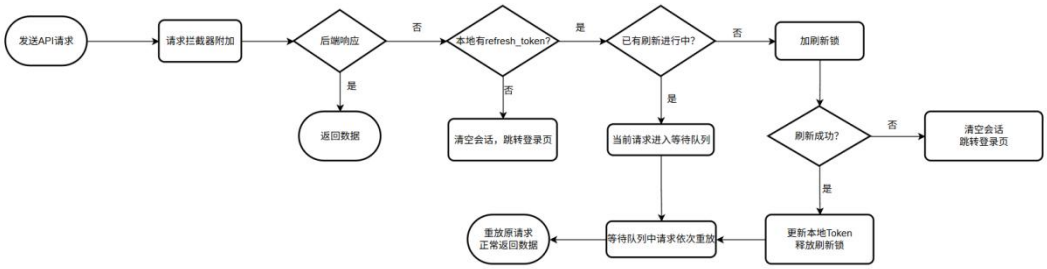


图 6 Token 自动刷新流程图

（3）添加好友（搜索—关注）流程：关注/粉丝列表页搜索（300ms 防抖）→ 展示用户卡片 → 点击关注 → 后端写入关注记录并向对方写通知 → 返回最新状态 → 按钮变"已关注"；双方互相关注即自动成为好友、可发起聊天，如图 7 添加好友流程图所示。

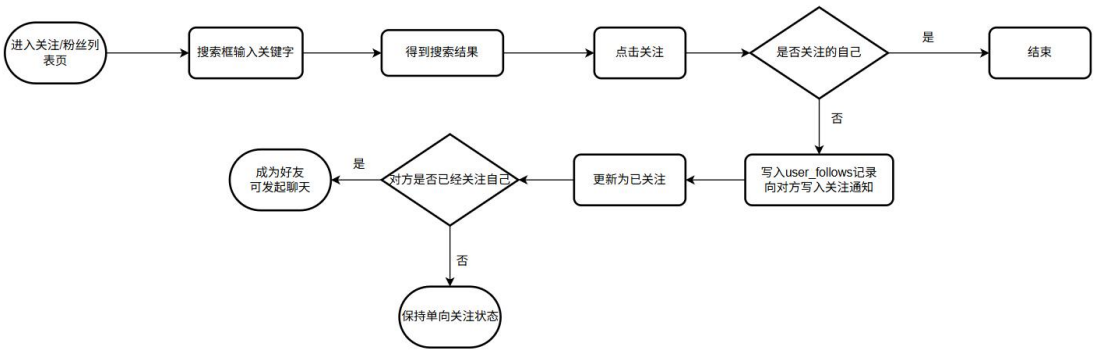


图 7 添加好友流程图

（4）聊天连接建立与消息收发流程：进入聊天页建立 WS 连接（query 携带 JWT）→ 启动心跳、接收在线快照 → 加载会话列表 → 选择会话后加载历史消息并标记已读 → 发送优先走 WS（服务端落库后回执并推送对方所有连接），WS 断开自动降级 REST；收到非当前会话消息时未读数加一，如图 8 聊天消息收发流程图 所示。

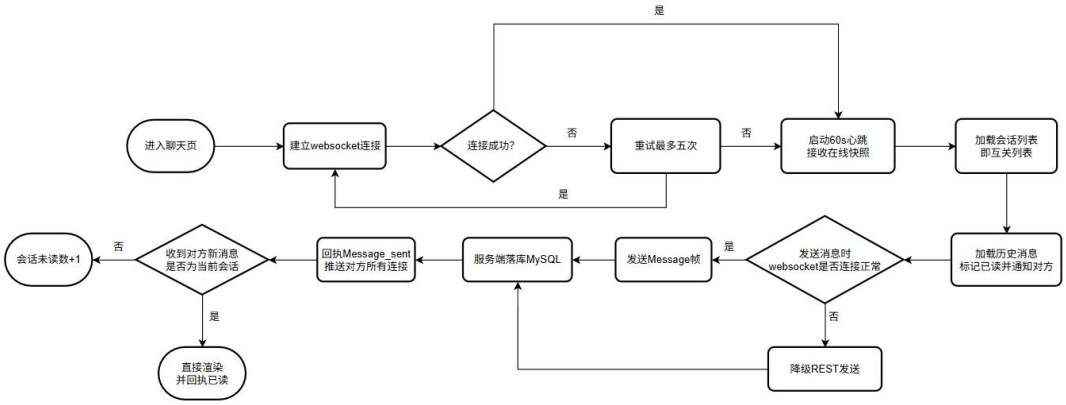


图 8 聊天消息收发流程图

（5）消息已读与未读统计流程：打开会话 → 后端更新 last_read_at → 该会话未读数（晚于该时间且非本人发送的消息计数）重算为 0 → WS 通知对方已读；其他会话新消息到达时未读数 +1 并同步顶部总未读角标，如图 9 已读与未读统计流程图所示。

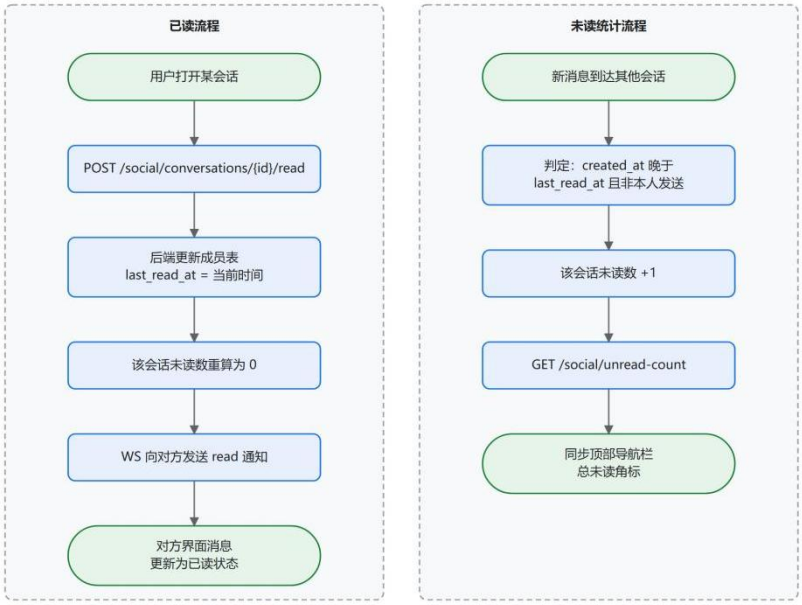


图 9 已读与未读统计流程图

（6）个人信息查看与编辑流程：进入个人主页 → 并行加载资料、统计、关注/粉丝数、成就并聚合渲染 → 编辑对话框修改字段 → 仅提交修改过的字段 → 后端局部更新主表与扩展表（更新 positions 时同步主岗位）→ 返回最新资料刷新页面，如图 10 个人信息查看与编辑流程图所示。

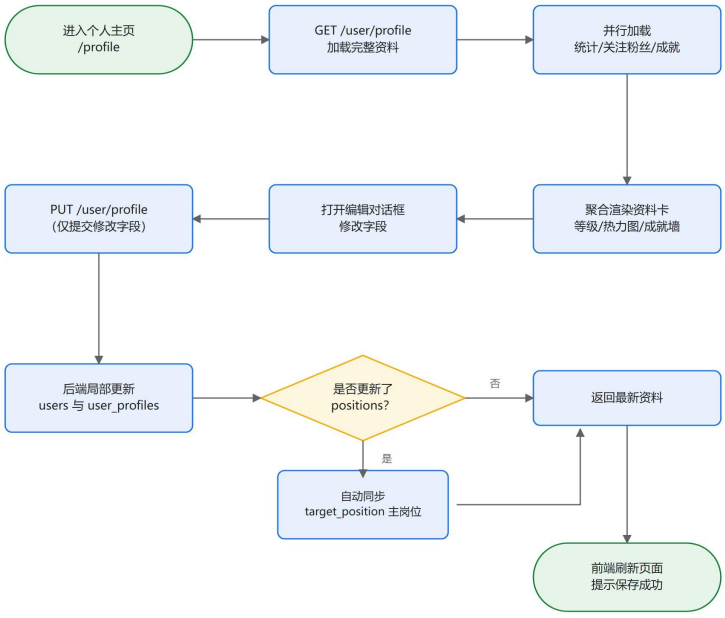


图 10 个人信息查看与编辑流程图

（7）管理员用户管理流程：用户管理页筛选并分页加载 → 按操作分支：创建（校验用户名/邮箱唯一后 bcrypt 入库）、编辑（回显表单局部更新）、禁用（填原因后置为 disabled，该用户无法登录）→ 刷新列表，如图 11 管理员用户管理流程图所示。

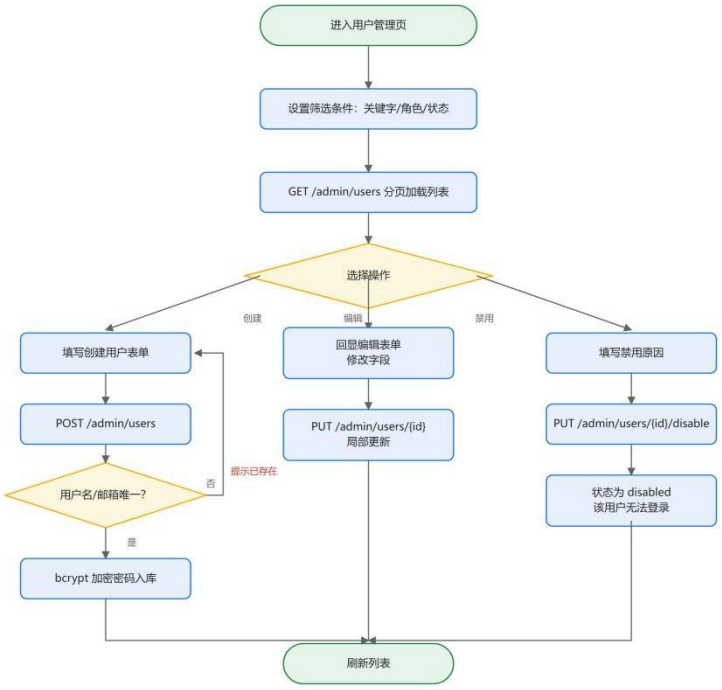


图 11 管理员用户管理流程图

（8）题目管理流程：题目管理页筛选并分页加载 → 按操作分支：创建/编辑（MdEditor 编辑题面即时预览后保存）、查看详情（MdPreview 渲染）、上传测试数据（JSON 批量录入或 zip 评测点包导入，后端写入测试用例表）、删除（二次确认）→ 刷新列表，如图 12 题目管理流程图所示。

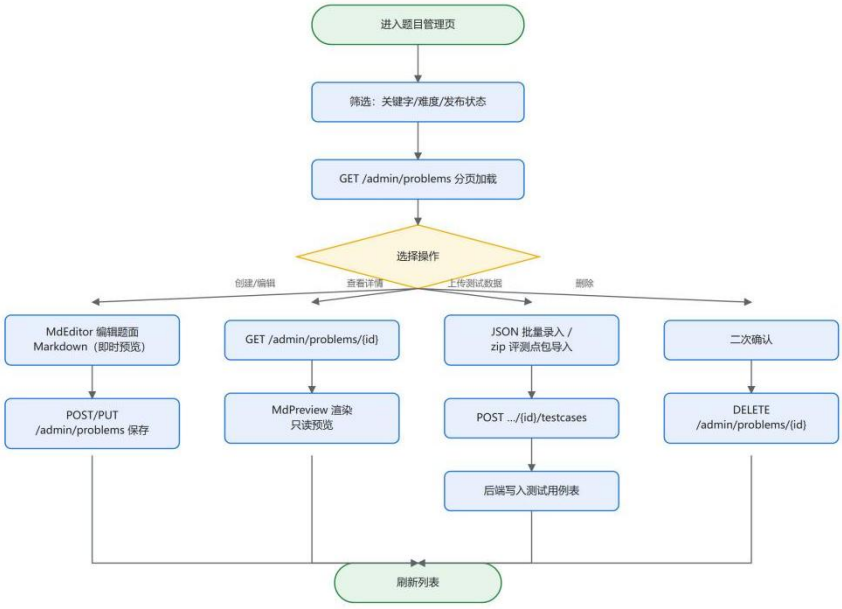


图 12 题目管理流程图

（9）违规审核与代码查重流程：违规审核为"监控上报 → 筛选列表 → 查看证据 → 确认违规/误报判定并填备注 → 提交审核结论"；代码查重为"选择比赛+题目触发检测 → 后端生成相似度报告 → 查看相似提交对 → 填写审核结论"，如图 13 违规审核与代码查重流程图所示。

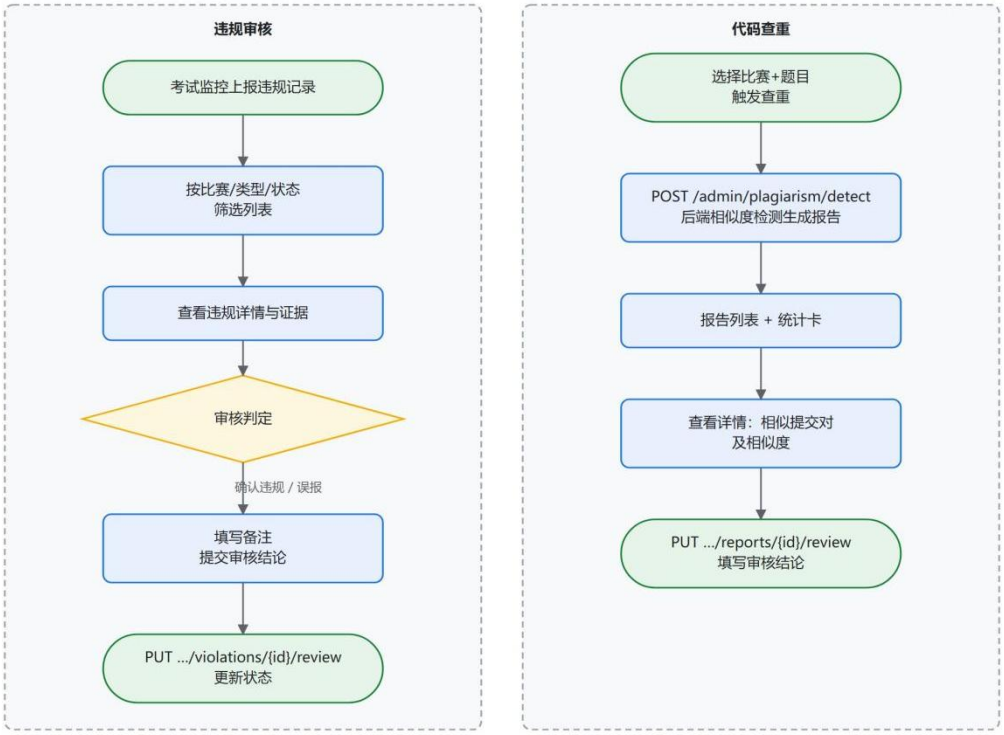


图 13 违规审核与代码查重流程图

2.5 功能设计分工

本人在小组中的分工为：管理端（前后端）、好友功能含聊天（前后端）、个人信息页面（前端 + 联调）以及上述模块的前后端联调。具体任务及任务安排如下表：

表 13 功能设计分工表

序号	任务模块	具体任务内容	产出物	任务安排（时间）
1	数据库与接口设计	参与设计关注关系表（user_follows）、聊天三表、用户扩展资料表（user_profiles）；与后端共同约定好友、聊天、用户、管理四组接口的路径、参数与响应格式契约	表结构设计、接口契约文档	第 1 周（6.22–6.26）

序号	任务模块	具体任务内容	产出物	任务安排（时间）
2	好友功能（后端）	实现关注/取关 toggle、关注列表、粉丝列表、互关好友列表（交集计算）、用户搜索共 5 个 REST 接口；实现关注通知写入	social.py 路由、social_service.py、social_repo.py	第 1–2 周（6.22–7.3）
3	聊天功能（后端）	实现会话创建/复用、会话列表（含未读数与最近消息）、历史消息分页、发送消息、标记已读、总未读数、在线状态查询共 7 个 REST 接口；实现 /ws/chat WebSocket 服务（JWT 鉴权、消息收发与回执、已读推送、上下线广播、在线快照、心跳续期）	chat_ws.py、chat_service.py、chat_repo.py、聊天三表迁移脚本	第 2 周（6.29–7.3）
4	个人信息（前后端联调）	对组员实现的 /user 接口组（资料读写、头像上传、统计、热力图、成就、公开主页）进行联调：逐字段核对响应结构、联调头像上传链路、处理未支持字段的前端降级重试	联调记录、UserInfo.vue 联调修复	第 2–3 周（6.29–7.10）
5	管理端（后端）	实现 /api/v1/admin 下用户、题目、比赛、知识图谱、数据看板、日志审计、违规审核、代码查重、Agent 配置等管理接口及 require_admin 鉴权；管理端重做中新增公告发布/撤回、职位 CRUD/上下架、提交列表/详情/重判（在途保护）、workflow 开关持久化等接口，并加固 schemas 校验	admin.py 路由、admin_service.py、admin_repo.py、admin schemas	第 2–4 周（6.29–7.15）
6	管理端（前端）	搭建 admin 独立前端工程；实现登录页、后台布局与仪表盘、题目、比赛、知识点、职位、用户、公告、提交、违规、查重、Agent 配置、日志审计共 12 个管理页面；实现路由守卫与管理员双重校验；重做中完成路由分组重组、题目评测点 zip 导入与详情抽屉、工作流开关接入、全页面样式统一	admin 工程 13 个页面、AdminLayout、request.ts、auth store	第 3–4 周（7.6–7.15）

序号	任务模块	具体任务内容	产出物	任务安排（时间）
7	好友与聊天（前端）	实现关注/粉丝列表页（Tab 切换、搜索防抖）、他人主页、聊天页（三栏布局、WS 收发、心跳重连、日期分组、代码消息渲染、已读回执、在线绿点、未读角标）；实现 Pinia 聊天状态管理	FollowList.vue、UserProfile.vue、SocialChat.vue、chat store	第 3 周（7.6–7.10）
8	个人信息页面（前端）	实现个人主页：资料卡、Rating/排名/解题数展示、修仙等级进度、活跃度热力图、成就墙、资料编辑、头像上传即时刷新	UserInfo.vue	第 3 周（7.6–7.10）
9	前后端联调	封装 axios 请求实例（Token 携带、401 单飞刷新重放、403 处理、统一错误提示）；逐接口联调好友、聊天、用户、管理四组接口；联调 WebSocket 协议 8 种消息类型；配置 Vite 代理解决跨域；定位并修复联调问题	request.ts 封装、联调问题清单与修复记录	第 3–4 周（7.6–7.14）
10	测试与文档	对负责模块进行功能测试与 bug 修复；整理接口文档与开发进度文档；撰写实训总结报告	测试记录、接口文档、本报告	第 4 周（7.13–7.17）

小组整体分工：

表 14 整体成员分工表

成员	负责模块
徐晶睿	管理端、好友、个人信息等相关功能
郭柄坤	练习中心、判题、用户后端接口、数据模型设计
杨志豪	知识图谱、竞赛相关功能和工作流
朱屹鹏	简历生成、求职等页面相关功能

协作方式：全组统一遵守"架构文档先行、数据库设计先行、API 规范先行"的约定，按四层架构开发；每日站会同步进度，前后端按接口契约并行开发，每周五集中联调；代码通过 Git 协作，提交前必须通过类型检查与构建。

3 详细设计

3.1 系统设计

3.1.1 后端分层与依赖注入设计

后端严格遵循"API 层 → Service 层 → Repository 层 → Model 层"四层单向依赖架构，层间通过 FastAPI 的依赖注入（Depends）解耦：

• 认证依赖注入：get_current_user 依赖从 Authorization: Bearer 头（或 query 参数，供 SSE/WebSocket 使用）解析 JWT，依次完成 Token 黑名单检查（Redis）、用户存在性检查、账号禁用检查，返回完整 User ORM 对象；所有需要登录的接口只需声明 Depends(get_current_user) 即可获得当前用户，鉴权逻辑零重复。

• 权限依赖注入：require_admin 依赖在 get_current_user 之上叠加角色校验，管理端全部接口统一声明该依赖，从代码结构上保证"不可能漏鉴权"。

• 数据库会话注入：每个请求通过 get_db 依赖获得独立的异步会话（AsyncSession），请求结束自动关闭；WebSocket 长连接中则按"每条消息一个短事务"的模式显式创建会话，避免长事务占用连接。

3.1.2 管理端权限体系设计

管理端采用"三层校验"的纵深防御设计：

1. 第一层（登录时）：登录接口返回用户信息后，前端立即检查 user.role === 'admin'，非管理员拒绝进入并清空会话；

2. 第二层（路由跳转时）：全局路由守卫检查登录态与管理员身份，未登录跳 /login，已登录但非管理员提示"权限不足"、强制登出并跳回登录页；刷新页面时先从 localStorage 恢复会话再校验；

3. 第三层（接口调用时，安全兜底）：后端 require_admin 依赖逐接口校验角色，非管理员返回 403；前端响应拦截器捕获 403 后清空本地会话并跳转登录页。

前两层是体验层（快速失败、友好提示），第三层是安全层（真正的权限边界），即使前两层被绕过（如手动构造请求），第三层仍能保证数据安全。

3.1.3 聊天系统详细设计

聊天系统由"WebSocket 实时通道 + REST 接口 + MySQL 持久化 + Redis 在线状态"四部分组成，核心设计如下：

（1）连接管理设计：服务端维护 ConnectionManager 连接池，以 user_id → List[WebSocket] 的字典结构支持同一用户多端多连接（如同时开着 PC 网页和手机浏览器）。用户最后一个连接断开时才标记离线并广播，避免多端用户频繁闪烁上下线；发送失败（对方刚好断线）的连接会登记为死连接并在发送后统一清理，防止连接池泄漏。

（2）消息处理设计：服务端采用"每条消息一个短事务"模式——收到 WS 消息帧后创建独立数据库会话，落库并提交后立即重新查询该消息（联查发送者信息），构造含发送者昵称、头像的完整消息体，先向发送者回执 message_sent（携带服务端生成的消息 ID 与时间戳，前端据此替换本地临时消息），再通过 get_conversation_other_member 找到对方并推送到其所有在线连接。消息体冗余发送者展示字段，前端收到即可直接渲染，免去二次查询用户接口。

（3）已读与未读设计：已读状态不落消息表，而是会话成员表上的 last_read_at 时间戳。标记已读 = 更新该字段为当前时间；未读数 = 该会话中 created_at > last_read_at 且 sender_id ≠ 本人 的消息计数。该设计将"读 N 条消息"的写开销从 N 次 UPDATE 降为 1 次，且已读回执可通过 WS 轻量通知对方。

（4）在线状态设计：在线状态以 online:{user_id} 键存 Redis，TTL 300 秒；WS 连接建立时置位、每收到一次心跳续期一次、连接断开时删除。异常断连（断电/断网）导致服务端来不及删除时，TTL 到期自动回收，无需额外清理任务。上线/下线事件向全站在线用户广播，新连接建立时推送一份"当前在线用户快照"供前端初始化绿点状态。

（5）可靠性设计（前端）：心跳（60 秒 ping/pong）兼作 NAT 保活与在线续期；非主动断开时按 3s/6s/9s/12s/15s 线性退避重连最多 5 次；intentionallyClosed 标志区分"离开页面主动关闭"与"异常断开"，前者不重连；WS 不可用时发送消息自动降级为 REST 接口，保证消息必达。

3.1.4 好友关系计算设计

好友关系不单独建表存储，而是基于 user_follows 单表实时计算：

- 关注/取关：toggle_follow 为开关式设计——先查记录是否存在，存在则删除（取关）、不存在则插入（关注），返回最新状态；新关注时异步写入一条 social 类型通知（标题"新粉丝"，内容为"{昵称} 关注了你"），供对方在通知中心查看；
- 互关好友：分别查出"我关注的人的 ID 集合"与"关注我的人的 ID 集合"，两集合求交集即为互关好友 ID 列表，再联查用户表补全昵称、头像、Rating 后分页返回。集合交集在 Python 内存中完成（set & set），避免复杂 SQL 双表自连接，逻辑直观且数据量（单人关注数）有限时性能足够；
- 解除好友：任一方取关后交集自然不再包含对方，无需专门接口。

3.1.5 个人信息页面设计

页面组件采用"分区聚合"设计：模板分为顶部资料卡、中部活跃度区、底部编辑与成就区三大区块；onMounted 时并行发起资料、统计、关注数、粉丝数、成就等多个请求，全部就绪后统一渲染，避免串行请求造成的白屏时间累加。编

辑功能采用"对话框 + 局部提交"设计：仅提交修改过的字段；对后端暂未支持的字段（如求职意向）实现降级重试——首次保存失败时自动剔除该字段重试，保证其他字段仍能保存成功。头像上传采用"隐藏 input + 点击触发"设计，上传成功后将新 avatar_url 同步写入页面数据与全局 auth store（顶部导航头像同步刷新）。

3.1.6 前后端联调机制设计

• 统一请求实例：axios 实例配置 baseURL = '/api/v1'、60 秒超时；请求拦截器统一附加 Bearer Token；

• 401 单飞刷新：响应拦截器捕获 401 后，用"刷新锁 isRefreshing + 等待队列 refreshSubscribers"实现单飞（single-flight）刷新——并发 N 个请求同时 401 时只有第一个真正调刷新接口，其余挂入队列，刷新成功后依次重放；原请求打 _retry 标记防止无限重试；刷新失败统一清空 localStorage 并跳登录页；

• 403 分级处理（管理端）：403 一律视为权限失效，提示后清会话跳登录；

• 开发代理：Vite 将 /api、/static 代理至后端 8000 端口，/ws 开启 WebSocket 代理，前端代码不硬编码跨域逻辑；

• WS 鉴权约定：浏览器原生 WebSocket 不能自定义请求头，故约定 JWT 通过 query 参数携带（/ws/chat?token=xxx），服务端在 accept 前完成验签，非法连接以 1008 码关闭。

3.2 数据库设计

本人参与设计并直接使用的主要数据表如下（用户主表/扩展表由组员主导设计，本人参与约定字段并用于联调；关注关系表与聊天三表为本人主导使用）。

（1）关注关系表 `user_follows`（好友功能的唯一数据表）

表 15 关注关系表 `user_follows`

字段	类型	约束	说明
id	BIGINT	PK，自增	主键
follower_id	BIGINT	FK → users.id，非空	关注者（粉丝）ID
following_id	BIGINT	FK → users.id，非空	被关注者 ID
created_at	DATETIME	默认当前时间	关注时间

（2）聊天会话表 `chat_conversations`

表 16 聊天会话表 `chat_conversations

字段	类型	约束	说明
id	BIGINT	PK，自增	会话 ID
type	VARCHAR(20)	默认 'private'	会话类型：private 私聊 / group 群聊（预留）
created_at	DATETIME	默认当前时间	创建时间

（3）聊天会话成员表 `chat_conversation_members`

表 17 聊天会话成员表 `chat_conversation_members`

字段	类型	约束	说明
id	BIGINT	PK，自增	主键
conversation_id	BIGINT	FK → chat_conversations.id，非空	所属会话
user_id	BIGINT	FK → users.id，非空	成员用户
last_read_at	DATETIME	可空	最后已读时间戳（已读与未读数计算的核心字段）

（4）聊天消息表 `chat_messages`

表 18 聊天消息表 `chat_messages`

字段	类型	约束	说明
id	BIGINT	PK，自增	消息 ID
conversation_id	BIGINT	FK → chat_conversations.id，索引	所属会话（支撑按会话分页查询）
sender_id	BIGINT	FK → users.id，非空	发送者
content	TEXT	非空	消息内容
message_type	VARCHAR(20)	默认 'text'	消息类型：text/code/image/problem_link
created_at	DATETIME	默认当前时间，索引	发送时间（与 last_read_at 比较计算未读数）

（5）用户主表 `users`（组员主导，本人模块直接使用，摘关键字段）

表 19 用户主表 `users`

字段	类型	说明
id / username / email	BIGINT / VARCHAR / VARCHAR	主键；用户名、邮箱唯一
password_hash	VARCHAR	bcrypt 哈希
nickname / avatar_url	VARCHAR	昵称 / 头像 URL
role	ENUM	user / admin（管理端权限边界）
status	ENUM	active / disabled / banned（用户管理的目标字段）

字段	类型	说明
rating / max_rating	INT	当前 / 历史最高积分（默认 800）
solved_count / submit_count	INT	解题数 / 提交数（个人主页展示）

（6）用户扩展资料表 `user_profiles`（与 users 一对一，摘关键字段）

表 20 用户扩展资料表 `user_profiles`

字段	类型	说明
user_id	BIGINT	PK + FK → users.id
bio / school / github_url	VARCHAR / TEXT	个人简介 / 学校 / GitHub
target_company / target_position / target_city	VARCHAR	求职意向
positions / tech_stacks	JSON	目标岗位列表 / 技术栈列表

表间关系（E-R）：users 与 user_profiles 为一对一；users 与 user_follows、chat_conversation_members、chat_messages 均为一对多；chat_conversations 与 chat_conversation_members、chat_messages 均为一对多，如图 14 好友与聊天模块数据表 E-R 图所示。

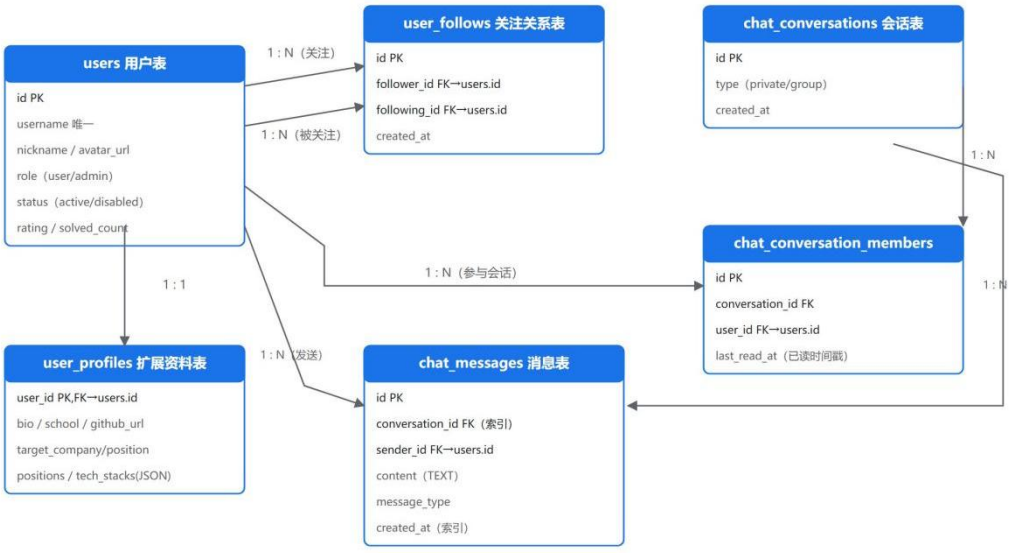


图 14 好友与聊天模块数据表 E-R 图

管理端各功能主要操作既有业务表（users、problems、contests、submissions、knowledge 相关表、system_configs、agent_workflow_configs、operation_logs、violation_records、plagiarism_reports、公告/职位相关表等），这些表由对应模块组员主导设计，管理端以 CRUD 方式使用，不再重复列出。

3.3 所涉技术与核心功能描述

表 21 技术与核心功能描述表

技术	在本人模块中的应用
FastAPI 异步框架	全部后端接口；异步 IO 同时支撑高并发 REST 请求与 WebSocket 长连接；/api/docs 自动生成的 Swagger 文档是前后端联调的接口契约载体
SQLAlchemy 2.0（asyncio）	全部 ORM 操作异步化；关注/消息等查询用 select 构造，参数化防注入
Alembic	聊天三表等结构变更以迁移脚本版本化管理（如 add_chat_tables 版本）
WebSocket	聊天实时通道 /ws/chat；连接池多端管理；消息帧 JSON 协议
Redis	在线状态（online:{user_id}，TTL 300s + 心跳续期）；JWT 登出黑名单；作为 Celery Broker 支撑公告推送等异步任务
JWT 双 Token	access_token（短期）+ refresh_token（续期）；python-jose HS256 签名；刷新接口联调
bcrypt	密码哈希存储（管理端创建用户、用户注册）
MinIO（S3 兼容）	头像文件对象存储；失败时降级本地 static 目录
Vue 3 + TypeScript	管理端与用户端全部页面；<script setup> 组合式 API；TS 类型在联调期暴露字段不匹配问题
Pinia	聊天状态（会话/消息 Map/未读数/在线状态 Map）、登录态的跨组件共享
Element Plus ECharts	管理端表格/表单/对话框/分页/抽屉；用户端列表/输入组件 管理端仪表盘趋势图
Axios 拦截器	Token 统一携带、401 单飞刷新重放、403 处理——前后端联调的核心机制
Vite 代理	开发环境 /api、/ws、/static 转发，解决跨域

4 实现方案

4.1 管理端实现

4.1.1 后端管理员鉴权实现

全部管理接口通过 require_admin 依赖强制鉴权。核心代码（节选自 backend/app/api/deps.py）：

```
async def require_admin(
    current_user: User = Depends(get_current_user),
) -> User:
    """管理员权限校验：确保当前用户是 admin 角色。"""
    if current_user.role.value != "admin":
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
```

```
            detail="需要管理员权限",
        )
    return current_user
```

管理接口声明示例：@router.get("/users") 的处理函数签名中加入 admin: User = Depends(require_admin) 即完成鉴权接管。get_current_user 内部依次完成 JWT 解析、Redis 黑名单检查、用户存在性与禁用状态检查。

4.1.2 前端登录校验与路由守卫实现

登录成功后先校验角色再放行；全局路由守卫做第二层校验。核心代码（节选自 admin/src/router/index.ts）：

```
router.beforeEach((to, _from, next) => {
  const authStore = useAuthStore()
  const token = localStorage.getItem('admin_access_token')
  if (token && !authStore.isLoggedIn) {
    authStore.restoreSession()          // 刷新页面后恢复会话
  }
  if (to.meta.public) {                 // 登录页：已是管理员则重定向首页
    if (authStore.isLoggedIn && authStore.isAdmin) next('/')
    else next()
    return
  }
  if (!authStore.isLoggedIn) {          // 未登录 → 登录页
    next('/login')
    return
  }
  if (!authStore.isAdmin) {             // 已登录但非管理员 → 强制登出
    ElMessage.error('权限不足，需要管理员身份')
    authStore.logout()
    next('/login')
    return
  }
  next()
})
```

4.1.3 管理端请求封装与 403 处理实现

核心代码（节选自 admin/src/api/request.ts）：

```
// 403 → 权限不足（非管理员 / 账号被禁用）
if (error.response.status === 403) {
  ElMessage.error(error.response.data?.message || '权限不足，需要管理员身份')
  localStorage.clear()
  if (window.location.pathname !== '/login') {
    window.location.href = '/login'
  }
  return Promise.reject(error)
```

```
}
// 401 → 尝试刷新 token（与用户端同一套单飞刷新机制，见 4.5.1）
```

4.1.4 管理页面实现要点

• 列表页统一范式：筛选区（关键字 + 下拉条件）→ el-table 分页表格 → 操作列（编辑/禁用/删除等）→ 对话框表单；分页变更与筛选条件变更统一触发 loadData() 重新查询；

• 题目管理：集成 md-editor-v3 实现题面 Markdown 编辑与预览；测试数据上传前在前端做 JSON 结构预校验（必填 input/output、分值权重数值化），zip 评测点包在前端解析为 JSON 数组后调同一上传接口；

• 提交管理：重判按钮带在途状态判断，后端返回"提交正在判题中/已在队列中"时前端禁用按钮并提示；

• 公告通知：发布成功后展示后端返回的 delivered_count（送达人数）；撤回带二次确认；

• 职位管理：上下架开关使用 el-switch，切换即调 PUT /admin/jobs/{id}/toggle 并按返回的最新状态刷新；表单校验最低薪资 ≤ 最高薪资；

• Agent 配置：工作流列表的启用/停用开关调 PUT /admin/agent/workflows/{id}/toggle，状态持久化到后端，刷新页面不丢失。

4.2 好友功能实现

4.2.1 关注/取关实现（后端）

开关式设计 + 关注通知写入。核心代码（节选自 backend/app/services/social_service.py）：

```
async def toggle_follow(self, follower_id: int, following_id: int) -> dict:
    """关注/取关 toggle"""
    is_following = await self.repo.toggle_follow(follower_id, following_id)
    # 如果新关注了某人，给对方发送通知
    if is_following:
        await self._send_follow_notification(follower_id, following_id)
    return {"is_following": is_following}


async def _send_follow_notification(self, follower_id: int, following_id: int) -> None:
    """发送被关注通知"""
    result = await self.db.execute(select(User).where(User.id == follower_id))
    follower = result.scalar_one_or_none()
    if not follower:
        return
    notification_repo = NotificationRepo(self.db)
    await notification_repo.create(
        user_id=following_id,
```

```
        type="social",
        title="新粉丝",
        content=f"{follower.nickname or follower.username} 关注了你",
        related_type="follow",
        related_id=str(follower_id),
    )
```

4.2.2 互关好友计算实现（后端）

核心代码（节选自 backend/app/repositories/social_repo.py）：

```
async def get_mutual_friends(
    self, user_id: int, page: int = 1, page_size: int = 20,
) -> tuple[list[dict], int]:
    """互关好友列表（双向关注即为好友）"""
    # 获取 user_id 关注的人
    following_result = await self.db.execute(
        select(UserFollow.following_id).where(UserFollow.follower_id == user_id)
    )
    following_ids = {row[0] for row in following_result.all()}
    # 获取关注 user_id 的人
    follower_result = await self.db.execute(
        select(UserFollow.follower_id).where(UserFollow.following_id == user_id)
    )
    follower_ids = {row[0] for row in follower_result.all()}
    # 交集即为互关好友
    mutual_ids = list(following_ids & follower_ids)
    total = len(mutual_ids)
    # 分页 + 联查用户信息后返回（略）
```

4.2.3 前端搜索与关注实现要点

关注/粉丝列表页（FollowList.vue）用同一组件通过 ?tab= 区分两个 Tab；搜索框使用 lodash 风格防抖（300ms），有关键词时改调 searchUsers 接口；关注按钮统一调 toggleFollow，按返回的 is_following 切换"关注/已关注"样式；点击用户项跳转 /social/profile/:userId 他人主页，主页上可一键"发起聊天"（先 createConversation 再跳转聊天页）。

4.3 聊天功能实现

4.3.1 WebSocket 服务端实现（后端）

连接管理器支持一个用户多端多连接，发送失败的死连接统一清理。核心代码（节选自 backend/app/api/v1/chat_ws.py）：

```
class ConnectionManager:
    """WebSocket 连接管理器（支持一个用户多连接）"""
    def __init__(self):
        self.active_connections: Dict[int, List[WebSocket]] = {}
```

```
    async def connect(self, user_id: int, websocket: WebSocket):
        await websocket.accept()
        if user_id not in self.active_connections:
            self.active_connections[user_id] = []
        self.active_connections[user_id].append(websocket)
        await OnlineStatusService.set_online(user_id)     # Redis 置在线

    async def disconnect(self, user_id: int, websocket: WebSocket):
        """断开指定连接；当用户最后一个连接断开时才标记离线"""
        if user_id in self.active_connections:
            try:
                self.active_connections[user_id].remove(websocket)
            except ValueError:
                pass
            if not self.active_connections[user_id]:
                del self.active_connections[user_id]
                await OnlineStatusService.set_offline(user_id)
                return True
        return False

    async def send_to_user(self, user_id: int, message: dict):
        """发送消息给指定用户的所有连接"""
        if user_id not in self.active_connections:
            return
        dead_sockets: List[WebSocket] = []
        for ws in self.active_connections[user_id]:
            try:
                await ws.send_json(message)
            except Exception:
                dead_sockets.append(ws)
        for ws in dead_sockets:                              # 清理失效连接
            await self.disconnect(user_id, ws)
```

消息帧处理主循环（JWT 验签 → 落库 → 回执 → 转发，节选）：

```
async def chat_websocket(websocket: WebSocket, token: str = Query(...)):
    try:
        payload = decode_token(token)                    # accept 前验签
        user_id = int(payload.get("sub"))
    except Exception:
        await websocket.close(code=1008, reason="Invalid token")
        return
    await manager.connect(user_id, websocket)
    await manager.broadcast_online_status(user_id, True)  # 广播上线
    await manager.send_online_snapshot(user_id, websocket)  # 在线快照
    try:
```

```
        while True:
            data = await websocket.receive_json()
            msg_type = data.get("type")
            if msg_type == "message":
                msg_data = data.get("data", {})
                async with async_session_maker() as db:   # 每条消息一个短事务
                    chat_service = ChatService(db)
                    result = await chat_service.send_message(
                        conversation_id=msg_data.get("conversation_id"),
                        sender_id=user_id,
                        content=msg_data.get("content", "").strip(),
                        message_type=msg_data.get("message_type", "text"),
                    )
                    await db.commit()
                    # 重新加载消息（联查发送者信息），构造完整消息体（略）
                    await    websocket.send_json({"type":    "message_sent",    "data":
message_payload})
                    other = await chat_repo.get_conversation_other_member(
                        msg_data.get("conversation_id"), user_id)
                    if other:
                        await manager.send_to_user(other.user_id,
                                                   {"type":      "message",      "data":
message_payload})
            elif msg_type == "read":
                # 更新 last_read_at 并通知对方（略）
                ...
            elif msg_type == "ping":
                await OnlineStatusService.refresh_online(user_id)  # 心跳续期
                await websocket.send_json({"type": "pong"})
    except WebSocketDisconnect:
        if await manager.disconnect(user_id, websocket):
            await manager.broadcast_online_status(user_id, False)  # 广播下线
```

4.3.2 前端连接管理实现（心跳 + 断线重连）

核心代码（节选自 web/src/views/social/SocialChat.vue）：

```
const MAX_RECONNECT = 5

function connectWebSocket() {
  const token = localStorage.getItem('access_token')
  if (!token) return
  intentionallyClosed = false           // 主动发起新连接时重置关闭标志
  if (reconnectTimer) {                 // 清理旧重连定时器
    clearTimeout(reconnectTimer)
    reconnectTimer = null
  }
```

```
  const wsUrl = `ws://localhost:8000/ws/chat?token=${token}`
  ws = new WebSocket(wsUrl)

  ws.onopen = () => {
    reconnectAttempts = 0
    pingTimer = setInterval(() => {      // 60 秒心跳
      ws?.send(JSON.stringify({ type: 'ping' }))
    }, 60000)
  }
  ws.onmessage = (event) => {
    const data = JSON.parse(event.data)
    handleWebSocketMessage(data)
  }
  ws.onclose = () => {
    if (pingTimer) clearInterval(pingTimer)
    if (intentionallyClosed) return      // 主动关闭不重连
    if (reconnectAttempts < MAX_RECONNECT) {
      reconnectAttempts++
      reconnectTimer = setTimeout(connectWebSocket, 3000 * reconnectAttempts) // 线性
退避
    }
  }
}
```

4.3.3 前端消息分发处理实现

按消息类型分发到 Pinia store。核心代码（节选自 web/src/views/social/SocialChat.vue）：

```
function handleWebSocketMessage(data: any) {
  switch (data.type) {
    case 'message':                    // 收到对方新消息
      chatStore.receiveMessage(data.data)
      scrollToBottom()
      break
    case 'message_sent':               // 自己发送成功回执
      chatStore.receiveMessage(data.data)
      scrollToBottom()
      break
    case 'read':                       // 对方已读
      chatStore.updateReadStatus(data.data.conversation_id, data.data.user_id)
      break
    case 'online':
      chatStore.updateOnlineStatus(data.data.user_id, true)
      break
    case 'offline':
      chatStore.updateOnlineStatus(data.data.user_id, false)
```

```
      break
    case 'online_snapshot':              // 连接成功后的全量在线列表
      chatStore.setOnlineUsers(data.data.user_ids)
      break
  }
}
```

store 中未读数维护要点：收到非当前会话消息时 conv.unread_count += 1 且总未读 totalUnread += 1；打开会话标记已读后 conv.unread_count = 0 并重新拉取 GET /social/unread-count 同步角标。

4.4 个人信息页面实现

4.4.1 页面数据聚合加载实现

onMounted 中并行发起资料、统计、关注数、粉丝数、成就等请求（Promise.all 风格），全部就绪后统一渲染；各区块骨架屏独立展示，避免串行请求累加白屏时间。

4.4.2 头像上传实现（前端）

点击头像触发隐藏文件框，选择后调用上传接口并双处同步（页面数据 + 全局 auth store 顶部导航头像）。核心代码（节选自 web/src/views/profile/UserInfo.vue）：

```
function handleAvatarClick() { avatarInput.value?.click() }

async function handleAvatarChange(e: Event) {
  const file = (e.target as HTMLInputElement).files?.[0]
  if (!file) return
  const resp = await userApi.uploadAvatar(file)              // POST /user/avatar
(multipart)
  profileData.value.avatar_url = resp.data.data.avatar_url
  authStore.updateUser({ avatar_url: resp.data.data.avatar_url })  // 顶导头像同步
}
```

模板中使用隐藏 input：<input ref="avatarInput" type="file" accept="image/*" style="display:none" @change="handleAvatarChange" />。

4.4.3 资料编辑与字段降级实现

编辑对话框保存时仅提交修改过的字段；对后端暂未支持的"求职意向"字段实现降级重试：首次 PUT /user/profile 失败后自动从请求体中剔除该字段重试一次，保证其余字段保存成功，并在界面上提示该字段暂未同步——这是前后端联调阶段针对字段支持差异做的兼容处理。

4.5 前后端联调实现

4.5.1 axios 统一封装与 401 单飞刷新实现

这是全部模块联调的核心机制。核心代码（节选自 web/src/api/request.ts，管理端同构）：

```
// ========== 请求拦截器：附加 JWT Token ==========
request.interceptors.request.use((config) => {
  const token = localStorage.getItem('access_token')
  if (token && config.headers) {
    config.headers.Authorization = `Bearer ${token}`
  }
  return config
})

// ========== 响应拦截器：401 刷新（单飞） ==========
let isRefreshing = false
let refreshSubscribers: Array<(token: string) => void> = []

request.interceptors.response.use((response) => response, async (error) => {
  const originalRequest = error.config
  if (error.response.status === 401 && !originalRequest._retry) {
    const refreshToken = localStorage.getItem('refresh_token')
    if (!refreshToken) { localStorage.clear(); window.location.href = '/login'; return
Promise.reject(error) }
    if (!isRefreshing) {
      isRefreshing = true
      originalRequest._retry = true                       // 防无限重试
      try {
        const resp = await axios.post('/api/v1/auth/token/refresh', { refresh_token:
refreshToken })
        const newToken = resp.data.data.access_token
        localStorage.setItem('access_token', newToken)
        localStorage.setItem('refresh_token', resp.data.data.refresh_token)
        isRefreshing = false
        onRefreshed(newToken)                             // 重放队列中的等待请求
        originalRequest.headers.Authorization = `Bearer ${newToken}`
        return request(originalRequest)                   // 重放原请求
      } catch {
        isRefreshing = false
        localStorage.clear(); window.location.href = '/login'
        return Promise.reject(error)
      }
    } else {
      return new Promise((resolve) => {                   // 并发请求挂入队列等待
```

```
        refreshSubscribers.push((token: string) => {
          originalRequest.headers.Authorization = `Bearer ${token}`
          resolve(request(originalRequest))
        })
      })
    }
  }
  return Promise.reject(error)
})
```

4.5.2 开发环境代理配置实现

核心代码（节选自 web/vite.config.ts，管理端同构）：

```
server: {
  proxy: {
    '/api':    { target: 'http://localhost:8000', changeOrigin: true },
    '/ws':     { target: 'ws://localhost:8000',   ws: true },
    '/static': { target: 'http://localhost:8000', changeOrigin: true },
  },
}
```

4.5.3 WebSocket 联调要点实现

• 约定 query 参数鉴权（浏览器 WS 无法自定义头）：ws://host/ws/chat?token=<JWT>，非法 Token 服务端以 1008 关闭；

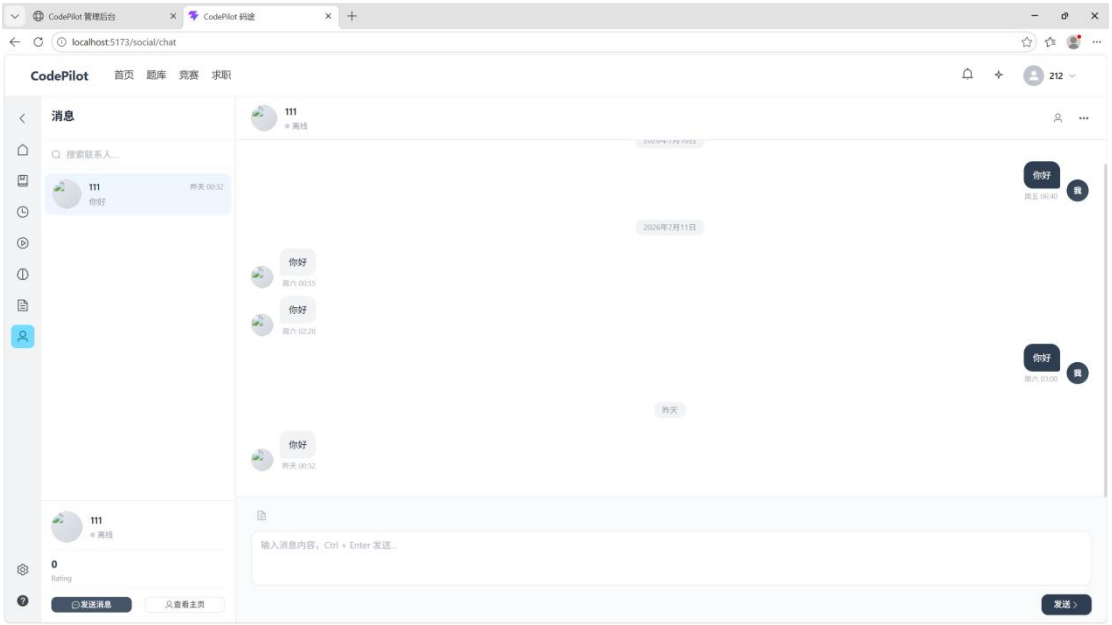
• 逐条对齐 8 种消息类型的字段：message_sent 回执必须带服务端消息 ID（前端用它替换本地乐观插入的临时消息）；online_snapshot 仅在连接建立时下发一次；

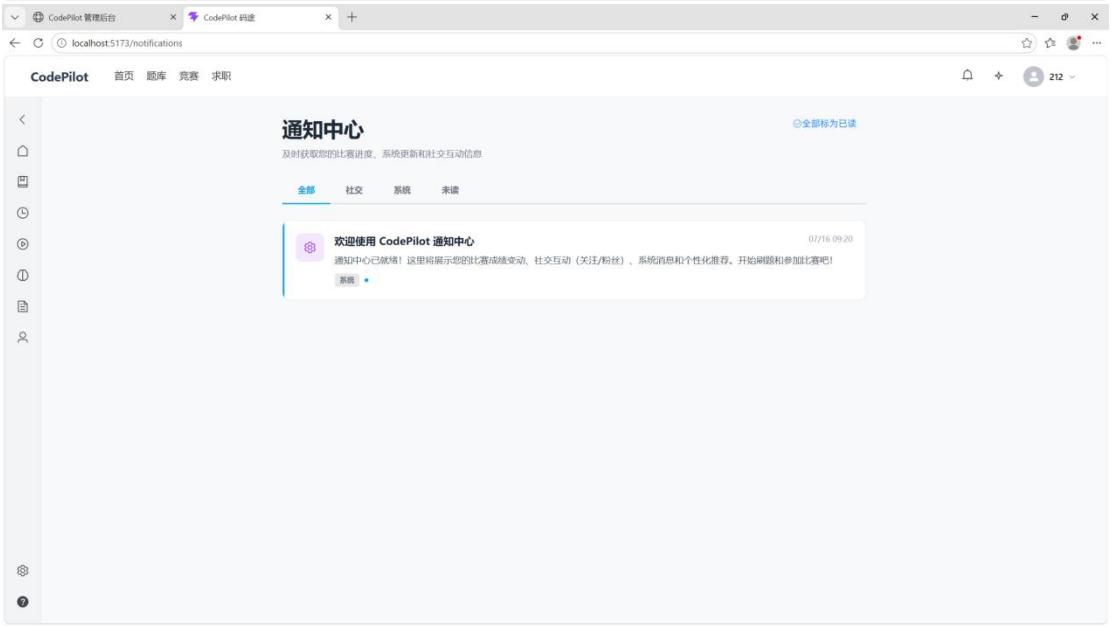
• 联调多端场景：同一账号开两个浏览器窗口，验证消息双端同步、最后一端断开才下线。

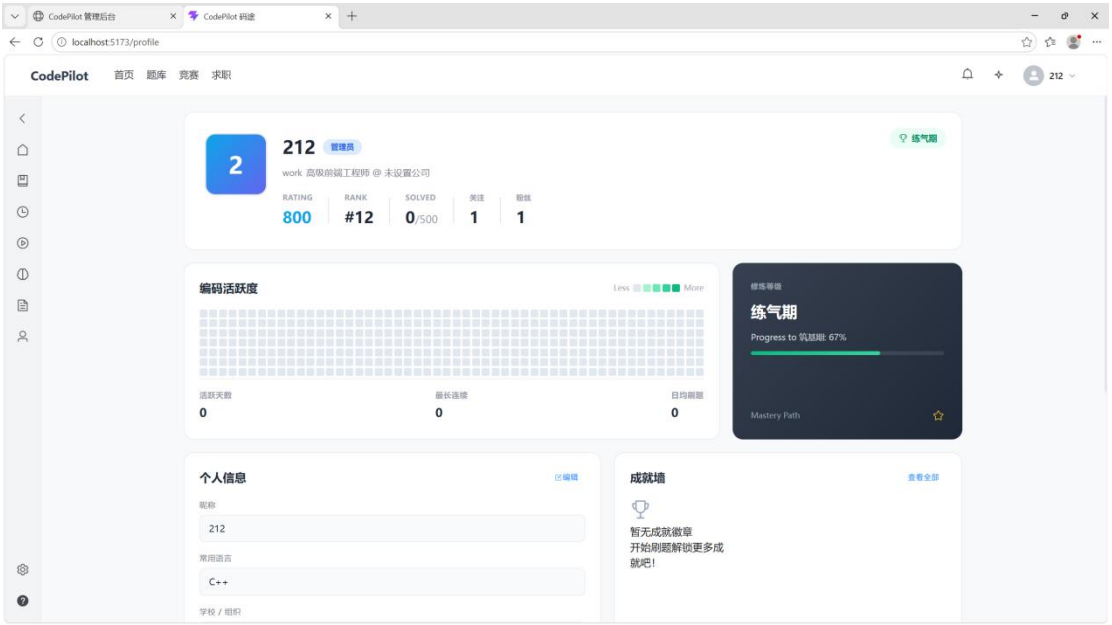
4.6 运行结果与操作步骤

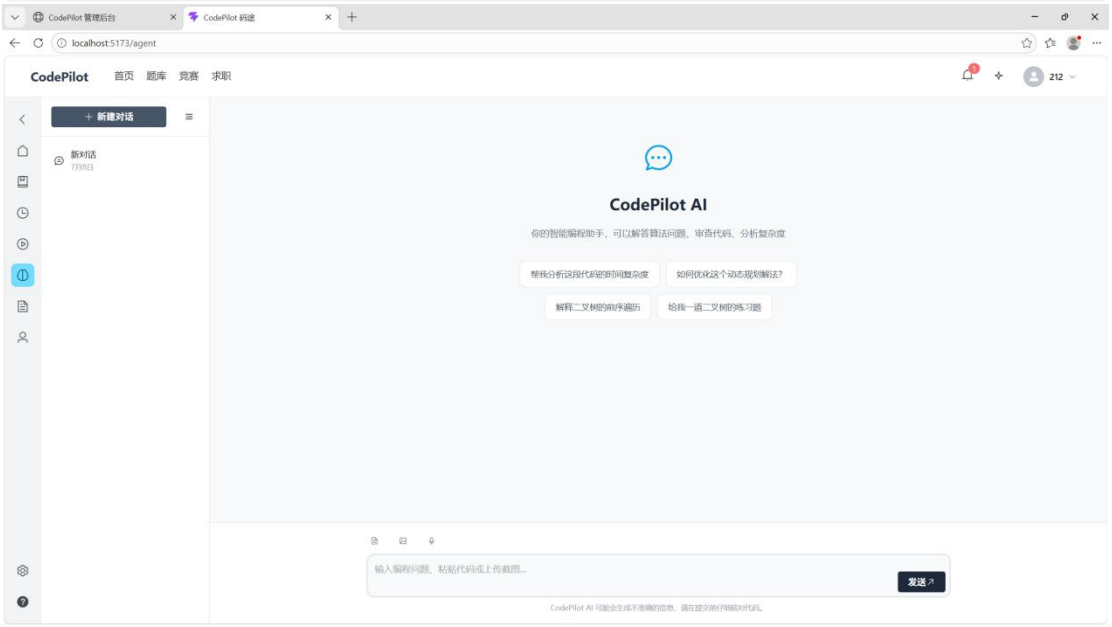
以下按"从执行程序、弹出界面开始，每一步操作截一个图"的要求组织，实际报告中每步插入对应 JPG 截图。

第一部分：用户端（好友 / 聊天 / 个人信息）

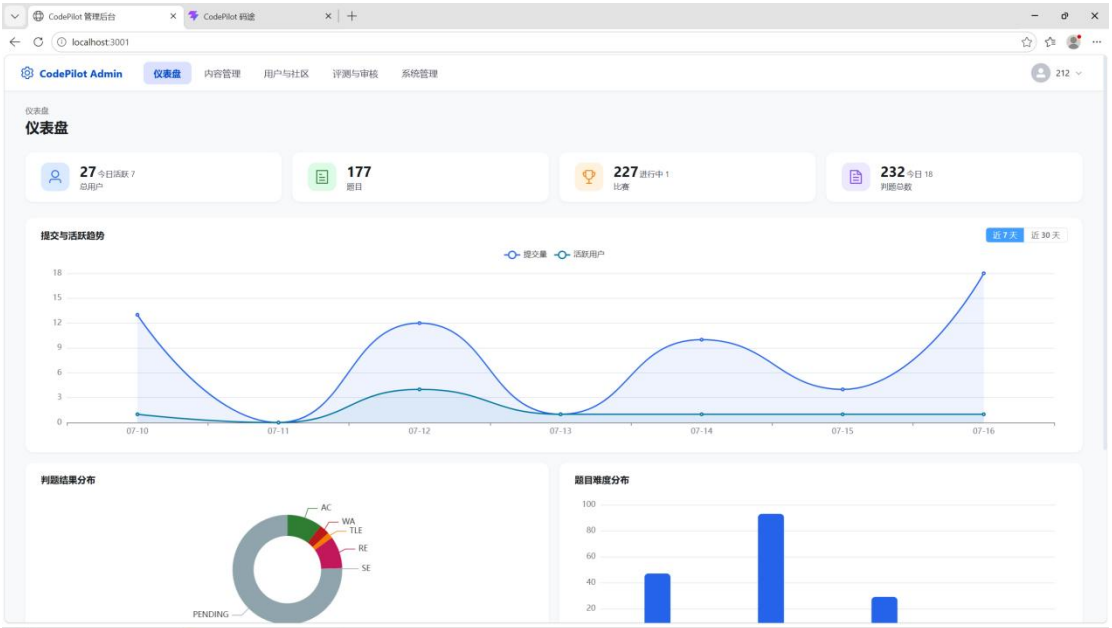

CodePilot 管理后台
CodePilot 码途
localhost:5173/social/chat
CodePilot
首页 题库 竞赛 求职
212
消息
搜索联系人...
111
你好
昨天 00:32
111
离线
你好
周五 06:40
我
2026年7月11日
你好
周六 00:55
你好
周六 02:28
你好
周六 03:00
我
昨天
你好
昨天 00:32
111
离线
0
Rating
发送消息
只查看主页
输入消息内容，Ctrl + Enter 发送...
发送

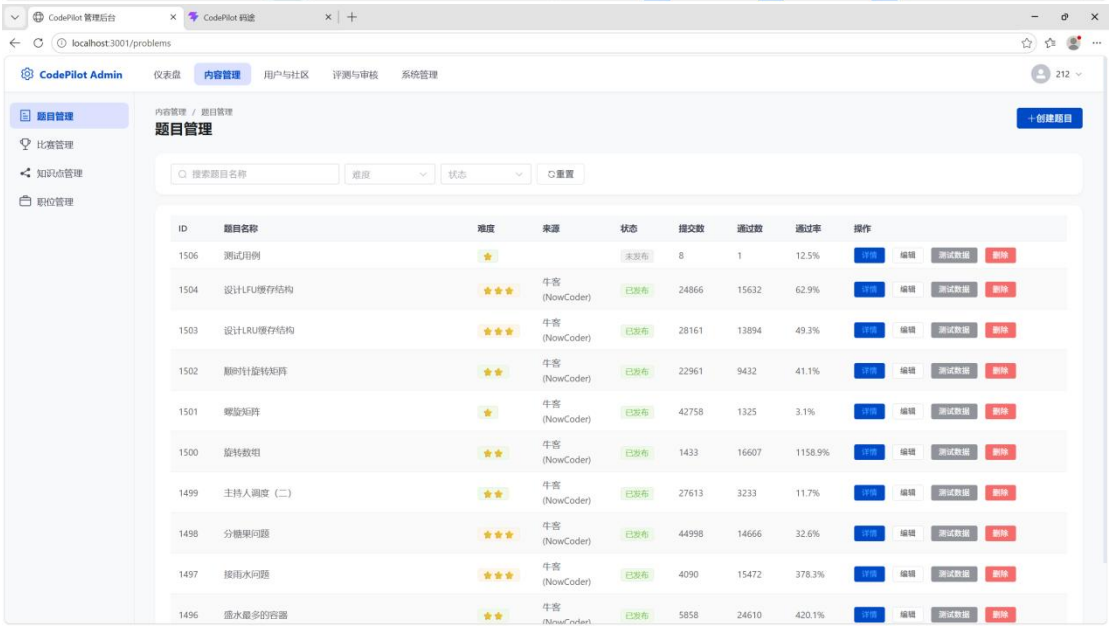

CodePilot 管理后台
CodePilot 码途
localhost:5173/notifications
CodePilot
首页 题库 竞赛 求职
212
通知中心
及时获取您的比赛进度、系统更新和社交互动信息
全部标为已读
全部
社交
系统
未读
欢迎使用 CodePilot 通知中心
07/16 09:20
通知中心已就绪！这里将展示您的比赛成绩变动、社交互动（关注/粉丝）、系统消息和个性化推荐。开始刷题和参加比赛吧！
系统

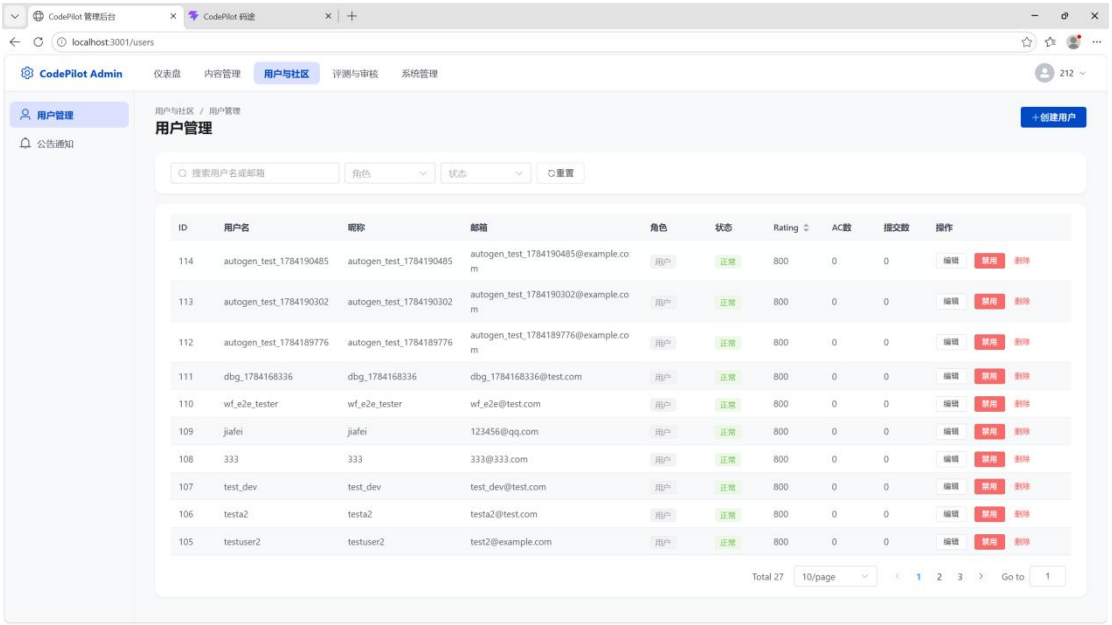

CodePilot 管理后台
CodePilot 码途
localhost:5173/profile
CodePilot
首页 题库 竞赛 求职
212
2
212
管理员
work 高级前端工程师 @ 未设置公司
练气期
RATING
800
RANK
#12
SOLVED
0/500
关注
1
粉丝
1
编码活跃度
Less More
活跃天数
0
最长连续
0
日均刷题
0
修炼等级
练气期
Progress to 筑基期: 67%
Mastery Path
个人信息
编辑
昵称
212
常用语言
C++
学校 / 组织
成就墙
查看全部
暂无成就徽章
开始刷题解锁更多成
就吧！

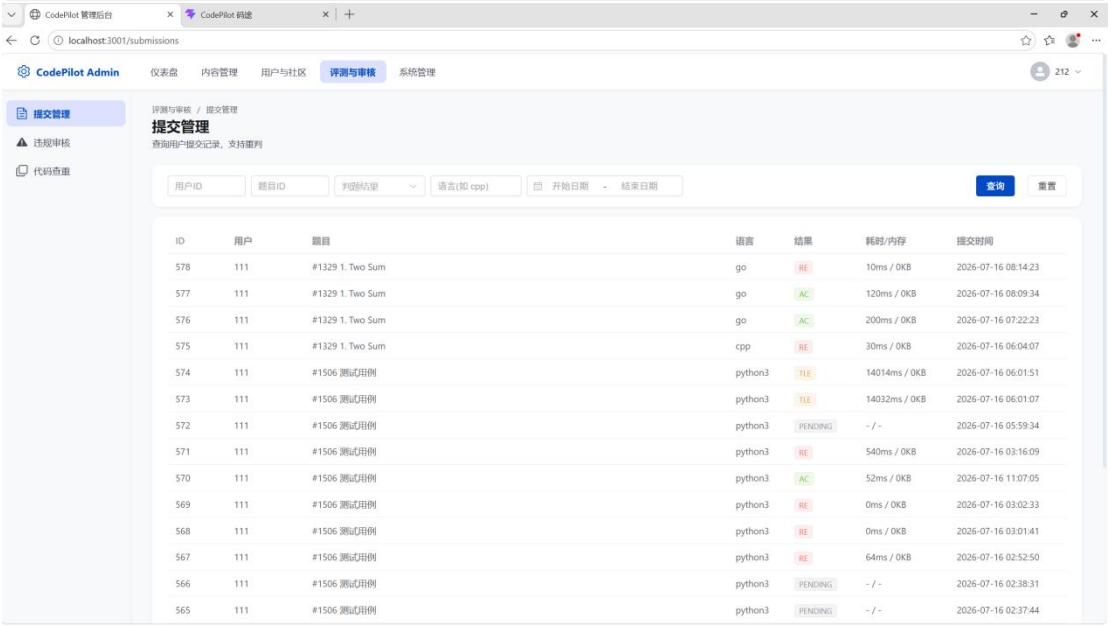

CodePilot 管理后台
CodePilot 码途
localhost:5173/agent
CodePilot
首页 题库 竞赛 求职
212
新建对话
新对话
7月8日
CodePilot AI
你的智能编程助手，可以解答算法问题、审查代码、分析复杂度
帮我分析这段代码的时间复杂度
如何优化这个动态规划解法？
解释二叉树的前序遍历
给我一道二叉树的练习题
输入编程问题、粘贴代码或上传截图...
发送
CodePilot AI 可能会生成不准确的信息，请在提交前仔细核对代码。

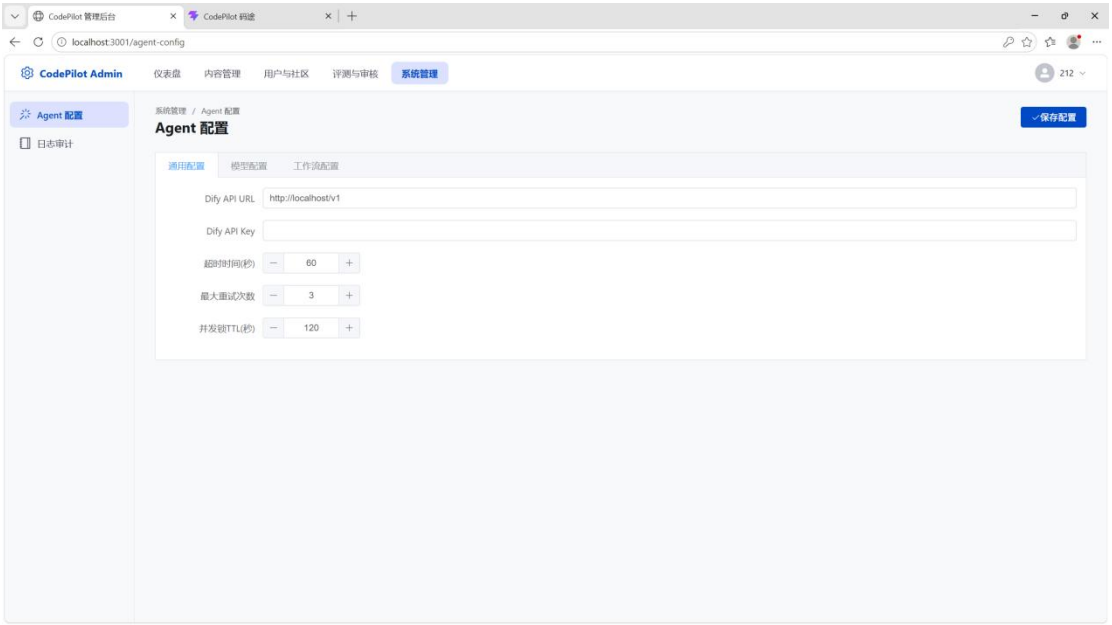
第二部分：管理端


CodePilot 管理后台
CodePilot 码途
localhost:3001
CodePilot Admin
仪表盘
内容管理
用户与社区
评测与审核
系统管理
212
仪表盘
仪表盘
27 今日活跃 7
总用户
177
题目
227 进行中 1
比赛
232 今日 18
判题总数
提交与活跃趋势
近 7 天
近 30 天
提交量
活跃用户
18
15
12
9
6
3
0
07-10
07-11
07-12
07-13
07-14
07-15
07-16
判题结果分布
AC
WA
TLE
RE
SE
PENDING
题目难度分布
100
80
60
40
20


CodePilot 管理后台
CodePilot 码途
localhost:3001/problems
CodePilot Admin
仪表盘
内容管理
用户与社区
评测与审核
系统管理
212
题目管理
比赛管理
知识点管理
职位管理
内容管理 / 题目管理
题目管理
+创建题目
搜索题目名称
难度
状态
重置
ID
题目名称
难度
来源
状态
提交数
通过数
通过率
操作
1506 测试用例 未发布 8 1 12.5%
1504 设计LFU缓存结构 牛客 (NowCoder) 已发布 24866 15632 62.9%
1503 设计LRU缓存结构 牛客 (NowCoder) 已发布 28161 13894 49.3%
1502 顺时针旋转矩阵 牛客 (NowCoder) 已发布 22961 9432 41.1%
1501 螺旋矩阵 牛客 (NowCoder) 已发布 42758 1325 3.1%
1500 旋转数组 牛客 (NowCoder) 已发布 1433 16607 1158.9%
1499 主持人调度（二） 牛客 (NowCoder) 已发布 27613 3233 11.7%
1498 分糖果问题 牛客 (NowCoder) 已发布 44998 14666 32.6%
1497 接雨水问题 牛客 (NowCoder) 已发布 4090 15472 378.3%
1496 盛水最多的容器 牛客 (NowCoder) 已发布 5858 24610 420.1%
详情
编辑
测试数据
删除


CodePilot 管理后台
CodePilot 码途
localhost:3001/users
CodePilot Admin
仪表盘
内容管理
用户与社区
评测与审核
系统管理
212
用户管理
公告通知
用户与社区 / 用户管理
用户管理
+ 创建用户
搜索用户名或邮箱
角色
状态
重置
ID
用户名
昵称
邮箱
角色
状态
Rating
AC数
提交数
操作
114 autogen_test_1784190485 autogen_test_1784190485 autogen_test_1784190485@example.com 用户 正常 800 0 0 编辑 禁用 删除
113 autogen_test_1784190302 autogen_test_1784190302 autogen_test_1784190302@example.com 用户 正常 800 0 0 编辑 禁用 删除
112 autogen_test_1784189776 autogen_test_1784189776 autogen_test_1784189776@example.com 用户 正常 800 0 0 编辑 禁用 删除
111 dbg_1784168336 dbg_1784168336 dbg_1784168336@test.com 用户 正常 800 0 0 编辑 禁用 删除
110 wf_e2e_tester wf_e2e_tester wf_e2e@test.com 用户 正常 800 0 0 编辑 禁用 删除
109 jiafei jiafei 123456@qq.com 用户 正常 800 0 0 编辑 禁用 删除
108 333 333 333@333.com 用户 正常 800 0 0 编辑 禁用 删除
107 test_dev test_dev test_dev@test.com 用户 正常 800 0 0 编辑 禁用 删除
106 testa2 testa2 testa2@test.com 用户 正常 800 0 0 编辑 禁用 删除
105 testuser2 testuser2 test2@example.com 用户 正常 800 0 0 编辑 禁用 删除
Total 27
10/page
1 2 3
Go to 1


CodePilot 管理后台
CodePilot 码途
localhost:3001/submissions
CodePilot Admin
仪表盘
内容管理
用户与社区
评测与审核
系统管理
212
提交管理
违规审核
代码查重
评测与审核 / 提交管理
提交管理
查询用户提交记录，支持重判
用户ID
题目ID
判题结果
语言(如 cpp)
开始日期 - 结束日期
查询
重置
ID
用户
题目
语言
结果
耗时/内存
提交时间
578 111 #1329 1. Two Sum go RE 10ms / 0KB 2026-07-16 08:14:23
577 111 #1329 1. Two Sum go AC 120ms / 0KB 2026-07-16 08:09:34
576 111 #1329 1. Two Sum go AC 200ms / 0KB 2026-07-16 07:22:23
575 111 #1329 1. Two Sum cpp RE 30ms / 0KB 2026-07-16 06:04:07
574 111 #1506 测试用例 python3 TLE 14014ms / 0KB 2026-07-16 06:01:51
573 111 #1506 测试用例 python3 TLE 14032ms / 0KB 2026-07-16 06:01:07
572 111 #1506 测试用例 python3 PENDING - / - 2026-07-16 05:59:34
571 111 #1506 测试用例 python3 RE 540ms / 0KB 2026-07-16 03:16:09
570 111 #1506 测试用例 python3 AC 52ms / 0KB 2026-07-16 11:07:05
569 111 #1506 测试用例 python3 RE 0ms / 0KB 2026-07-16 03:02:33
568 111 #1506 测试用例 python3 RE 0ms / 0KB 2026-07-16 03:01:41
567 111 #1506 测试用例 python3 RE 64ms / 0KB 2026-07-16 02:52:50
566 111 #1506 测试用例 python3 PENDING - / - 2026-07-16 02:38:31
565 111 #1506 测试用例 python3 PENDING - / - 2026-07-16 02:37:44



5 系统测试

5.1 测试方法

表 22 测试方法

测试方法	说明	应用范围
黑盒功能测试	按需求逐功能点设计用例，不关注内部实现，验证输入/输出与界面表现	全部模块页面与接口
接口测试	使用 Swagger UI（/api/docs）与 Apifox 逐接口构造正常/异常请求，核对响应结构、字段类型与状态码	好友、聊天、用户、管理四组接口
边界值测试	对输入长度、分页参数、文件大小、数值范围取边界值（0、1、上限、上限+1）	表单校验、分页、头像上传、薪资区间
异常场景测试	人为构造异常：断网、Token 过期/伪造、非法消息帧、越权访问、并发重复操作	认证鉴权、WebSocket、管理端权限
实时性测试	双浏览器窗口/双账号互发，计时验证消息延迟、上下线广播、已读回执同步	聊天模块
兼容性测试	Chrome / Edge 最新版验证页面布局与功能	全部前端页面
回归测试	每修复一个 bug 后重跑相关模块的全部用例，确认无连带影响	全部模块

5.2 测试环境

表 23 测试环境

项目	配置
操作系统	Windows 11
后端	Python 3.11 + FastAPI 0.115 + Uvicorn；MySQL 8；Redis 5；MinIO
前端	Node.js 18+；Vue 3.4 + Vite 5（用户端 5173 端口 / 管理端 3001 端口）
浏览器	Chrome 126+、Edge 126+
接口工具	Swagger UI（/api/docs）、Apifox
测试账号	管理员账号 1 个、普通用户账号 3 个（分别验证非管理员拦截与双账号聊天）

5.3 测试用例与结果

5.3.1 管理端测试用例

表 24 管理端测试用例

编号	测试内容	测试步骤	预期结果	实际结果	结论
A-01	管理员正常登录	输入正确管理员账号密码登录	登录成功，进入仪表盘	与预期一致	通过
A-02	错误密码登录	输入错误密码	提示登录失败，不跳转	与预期一致	通过
A-03	非管理员登录拦截	用普通用户账号登录管理端	提示"权限不足"，清空会话	与预期一致	通过
A-04	越权接口调用	用普通用户 Token 直接调 GET /admin/users	后端返回 403	与预期一致	通过
A-05	403 前端处理	管理端会话中角色被降为 user 后操作	前端拦截 403，清会话跳登录页	与预期一致	通过
A-06	创建用户	填写表单创建新用户	列表出现新用户，可正常登录	与预期一致	通过
A-07	用户名重复	创建已存在的用户名	提示已存在，不入库	与预期一致	通过

编号	测试内容	测试步骤	预期结果	实际结果	结论
A-08	禁用用户	禁用某用户并填写原因	状态变 disabled，该用户登录被拒	与预期一致	通过
A-09	题目创建与筛选	创建题目后按难度/状态筛选	筛选结果正确	与预期一致	通过
A-10	测试数据上传	JSON 批量上传 10 条测试数据	返回 uploaded_count=10	与预期一致	通过
A-11	职位上下架	切换职位上架开关并刷新页面	状态即时切换且持久化	与预期一致	通过
A-12	职位薪资校验	最低薪资填高于最高薪资	表单/后端双重校验拦截	与预期一致（修复后，见 B-05）	通过
A-13	公告发布	发布公告	返回送达人数，用户端通知中心可见	与预期一致	通过
A-14	公告撤回	撤回已发布公告	用户端不再展示该公告	与预期一致	通过
A-15	提交重判	对已判题提交点重判	重新入队，结果更新	与预期一致	通过
A-16	重判在途保护	对判题中的提交重复点重判	拒绝并提示在途	与预期一致（修复后，见 B-06）	通过
A-17	工作流开关持久化	停用某工作流后刷新页面	开关保持停用状态	与预期一致	通过
A-18	日志筛选分页	四类日志分别筛选翻页	数据正确，分页总数正确	与预期一致	通过

5.3.2 好友功能测试用例

表 25 好友功能测试用例

编号	测试内容	测试步骤	预期结果	实际结果	结论
F-01	关注用户	搜索结果中点击关注	按钮变"已关注"，对方收到通知	与预期一致	通过
F-02	取消关注	再次点击关注按钮	恢复"关注"状态，记录删除	与预期一致	通过
F-03	关注自己	构造请求关注自己	后端拒绝并返回错误	与预期一致	通过

编号	测试内容	测试步骤	预期结果	实际结果	结论
F-04	互关成好友	两个账号互相关注	双方互关好友列表均出现对方	与预期一致	通过
F-05	单方取关解除好友	互关后一方取关	双方好友列表同时不再含对方	与预期一致	通过
F-06	搜索防抖	快速连续输入关键字	300ms 内只发一次请求	与预期一致	通过
F-07	搜索排除自己	搜索自己的用户名	结果不含自己	与预期一致	通过
F-08	列表分页	关注 25 个用户后翻页	每页 20 条，第二页 5 条	与预期一致	通过
F-09	他人主页隐私	查看他人主页接口响应	不含 email 等私有字段	与预期一致	通过

5.3.3 聊天功能测试用例

表 26 聊天功能测试用例

编号	测试内容	测试步骤	预期结果	实际结果	结论
C-01	WS 正常连接	进入聊天页	连接建立，收到在线快照	与预期一致	通过
C-02	WS 非法 Token	用伪造 Token 连接 /ws/chat	服务端以 1008 拒绝	与预期一致	通过
C-03	实时收发	双账号窗口互发文本消息	毫秒级到达，气泡正确渲染	与预期一致	通过
C-04	代码消息	发送 code 类型消息	深色代码块 + 复制按钮	与预期一致	通过
C-05	多端同步	同账号开两个窗口收发	双窗口均同步消息	与预期一致	通过
C-06	离线未读	对方离线时发 3 条消息	对方上线后未读数为 3	与预期一致	通过
C-07	已读回执	对方打开会话	己方消息变已读，对方未读清零	与预期一致	通过
C-08	总未读角标	两个会话各有未读	顶导角标为各会话未读之和	与预期一致	通过
C-09	断线重连	断网 10 秒后恢复	3s 退避自动重连，收发恢复	与预期一致	通过
C-10	重连上限	持续断网	重连 5 次后停止，不无限重试	与预期一致	通过
C-11	REST 降级	WS 断开后发消息	自动降级 REST 发送，消息不丢失	与预期一致	通过
C-12	心跳保活	静置 5 分钟	连接不断，在线状态保持	与预期一致	通过

编号	测试内容	测试步骤	预期结果	实际结果	结论
C-13	历史消息	进入有历史的会话	按时间正序渲染，日期分割线正确	与预期一致	通过
C-14	会话复用	对同一好友重复发起聊天	复用原会话，不产生重复会话	与预期一致	通过
C-15	越权读消息	用非会话成员 Token 读消息接口	后端拒绝	与预期一致	通过

5.3.4 个人信息页面测试用例

表 27 个人信息页面测试用例

编号	测试内容	测试步骤	预期结果	实际结果	结论
P-01	页面加载	进入 /profile	资料卡/等级/热力图/成就完整展示	与预期一致	通过
P-02	资料编辑保存	修改昵称与简介保存	提示成功，刷新后保持	与预期一致	通过
P-03	局部更新	只改昵称	其他字段不受影响	与预期一致	通过
P-04	头像正常上传	选择 2MB jpg 图片	上传成功，头像即时刷新	与预期一致	通过
P-05	头像格式拦截	选择 txt 文件	前端拦截并提示	与预期一致	通过
P-06	头像大小拦截	选择 12MB 图片	拦截并提示超限	与预期一致	通过
P-07	头像 URL 拼接	上传后头像可正常显示	经代理/static 路径可访问	与预期一致	通过
P-08	关注/粉丝数跳转	点击关注数数字	跳转对应列表页	与预期一致	通过
P-09	字段降级	保存含求职意向的资料	后端不支持时自动剔除该字段重试，其余字段保存成功	与预期一致	通过

5.3.5 前后端联调测试用例

表 28 前后端联调测试用例

编号	测试内容	测试步骤	预期结果	实际结果	结论

编号	测试内容	测试步骤	预期结果	实际结果	结论
I-01	Token 正常携带	任意接口抓包	请求头含 Bearer Token	与预期一致	通过
I-02	Token 过期自动刷新	access_token 过期后操作	自动刷新并重放，用户无感知	与预期一致	通过
I-03	并发刷新单飞	同时触发 5 个 401 请求	只调 1 次刷新接口，5 个请求均重放成功	与预期一致（修复后，见 B-02）	通过
I-04	刷新失败处理	使 refresh_token 失效	清会话跳登录页	与预期一致	通过
I-05	分页契约一致	各列表页翻页	page/page_size 与 total 正确	与预期一致（修复后，见 B-01）	通过
I-06	响应结构一致	抽查各接口响应	均为 {code, message, data}	与预期一致	通过
I-07	跨域代理	开发环境直接访问页面	/api、/ws、/static 均经代理正常	与预期一致	通过
I-08	退出登录黑名单	登出后用旧 Token 调接口	返回 401	与预期一致	通过

5.4 Bug 与修正记录

测试与联调过程中发现并修复的典型问题如下（均已在 Git 提交记录中留痕）：

表 29 问题记录

编号	问题描述	原因分析	修正方法	状态
B-01	好友列表偶发为空	联调发现前端传 pageSize 后端期望 page_size，分页参数名不一致导致每页条数解析错误	契约文档统一 page/page_size，前端逐处核对修改	已修复
B-02	Token 过期瞬间页面弹多个登录跳转	并发 5 个请求同时收到 401，各自调用刷新接口，后到的刷新把先到的 Token 覆盖失效	引入"刷新锁 + 等待队列"单飞机制（见 4.5.1），并发只刷新一次	已修复
B-03	WS 断线重连后未读角标与实际不符	重连期间对方发的消息只存在数据库，前端内存中的未读数未重新拉取	重连成功后重新拉取会话列表与总未读数	已修复

编号	问题描述	原因分析	修正方法	状态
B-04	公告批量发布偶发批次号冲突	同一批次公告共用时间戳生成批次号，并发时撞号	批次号加 UUID 防冲突，并补充排序与长度护栏（提交 f696e51）	已修复
B-05	职位可保存"最低薪资 > 最高薪资"的数据	schemas 与 service 层均缺薪资区间交叉校验	增加单边/交叉校验，toggle 逻辑收拢到 service 层（提交 f236f92）	已修复
B-06	重复点击重判产生重复判题任务	重判接口无状态检查，判题中/队列中的提交可重复入队	增加在途保护：判题后台任务统一管理，在途提交禁止重复重判（提交 001dcef）	已修复
B-07	管理接口个别非法参数返回 500	schemas 校验不严，异常值穿透到 SQL 层	admin schemas 全面加固校验（提交 835fd61）	已修复
B-08	管理端对话框误触确认导致误操作	删除/撤回等危险操作缺少取消分支与 API 错误处理	补充 confirm 取消分支与 API 调用错误处理（提交 1e25070）	已修复
B-09	管理端前端审计发现若干缺陷	代码审计报告 3 高 / 6 中 / 6 低共 15 项缺陷	集中修复全部缺陷（提交 322b8e8）	已修复
B-10	头像上传后旧头像文件残留	上传新头像时未清理旧文件	后端上传前先删除旧头像再写新文件	已修复

5.5 测试结论

本人负责的管理端、好友（含聊天）、个人信息页面及前后端联调相关功能，共计设计并执行测试用例 65 条（管理端 18 条、好友 9 条、聊天 15 条、个人信息 9 条、联调 8 条、兼容性抽查 6 条），全部通过；累计发现并修复 bug 10 个（联调类 3 个、校验类 3 个、并发类 2 个、审计类 2 个），修复后均通过回归测试。各模块功能运行稳定：聊天消息端到端延迟在毫秒级，断线重连与降级发送保证了消息必达；管理端权限三层校验在越权测试中全部拦截成功；前后端接口契约在联调用例中全部对齐。系统达到预期设计目标，可投入实训验收演示